VECTOR

MICROSAR Classic CAN Driver

Technical Reference

NXP Semiconductors

ARM32

FlexCAN4

Version 1.09.00

Authors	visrli, visnaj
Status	Released

1 Document Information

1.1 History

Author	Date	Version	Remarks
visrli	2019-12-04	1.00.00	Initial creation
visrli	2020-01-29	1.01.00	Support of HW features EXTRA FD RAM, Enhanced Bit Timing
visrli	2020-05-04	1.01.01	Change description of Protected Area, Register Protection. Add S32K2TV/S32K3XX derivatives
visnaj	2021-01-12	1.02.00	Correct S32K344 to S32K3XX
visrli	2021-06-09	1.03.00	Add DIAB compiler
visrli	2021-07-21	1.04.00	S32R4XX support Security Event Reporting support
visrli	2021-08-12	1.05.00	KW45 support
visrli	2022-04-22	1.06.00	SAF85XX support
visrli	2022-06-02	1.07.00	MWCT2xx support S32K341, S32K3x8, S32K38x, S32K39x support S32G3xx support S32K3xx support clarification
visrli	2022-11-03	1.08.00	NCJ29D6 support ARM6 compiler support
visrli	2023-03-23	1.09.00	LLVM DIAB compiler support

1.2 Reference Documents

No.	Title	Version
[1]	AUTOSAR_SWS_CAN_DRIVER.pdf	4.0.0
[2]	AUTOSAR_BasicSoftwareModules.pdf	V1.0.0
[3]	AUTOSAR_SWS BSW Scheduler	V1.1.0
[4]	AUTOSAR_SWS_CAN_Interface.pdf	5.0.0
[5]	AN-ISC-8-1118 MICROSAR Classic BSW Compatibility Check	V1.0.0
[6]	ISO DIS 11898-1	2015

1.3 Scope of the Document

This document describes the functionality, API and configuration of the MICROSAR Classic CAN Driver as specified in [1]. The CAN Driver is a hardware abstraction layer with a standardized interface to the CAN Interface layer.



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector’s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1 Document Information 2
1.1 History 2
1.2 Reference Documents 2
1.3 Scope of the Document 2

2 Hardware Overview 7

3 Hardware Software Interface (HSI) 8

4 Introduction 9
4.1 Architecture Overview 10

5 Functional Description 11
5.1 Features 11
5.2 Initialization 14
5.3 Communication 14
5.4 States / Modes 17
5.5 Re-Initialization 19
5.6 CAN Interrupt Locking 19
5.7 Main Functions 19
5.8 Error Handling 20
5.9 Hardware Specific 27
5.10 Virtual Addressing 28

6 Integration 29
6.1 Scope of Delivery 29
6.2 Include Structure 30
6.3 Critical Sections 30
6.4 Compiler Abstraction and Memory Mapping 32
6.5 Hardware Specific Hints 34

7 API Description 37
7.1 Interrupt Service Routines provided by CAN 37
7.2 Services provided by CAN 39
7.3 Services used by CAN 66

8 Configuration 68
8.1 Pre-Compile Parameters 68
8.2 Link-Time Parameters 69
8.3 Post-Build Parameters 69

8.4 Configuration with da DaVinci Configurator 70

9 Glossary and Abbreviations 71

9.1 Glossary 71

9.2 Abbreviations 71

10 Contact 72

Illustrations

Figure 4-1 AUTOSAR architecture 10
Figure 4-2 Interfaces to adjacent modules of the CAN 10
Figure 6-1 Include Structure (AUTOSAR) 30

Tables

Table 2-1 Supported Hardware Overview 7
Table 5-1 Supported features 13
Table 5-2 Hardware mailbox layout 15
Table 5-3 Errors reported to DET 20
Table 5-4 API from which the Errors are reported 21
Table 5-5 Overrun/Overwrite state overview 22
Table 5-6 Critical Loops 23
Table 5-7 Uncritical Loops 24
Table 5-8 Hardware Loop Check 24
Table 5-9 Security Events 26
Table 6-1 Static files 29
Table 6-2 Generated files 29
Table 6-3 Critical Section Codes 32
Table 6-4 Compiler abstraction and memory mapping 33
Table 6-5 Hardware Controller – Area Defines 34
Table 6-6 Protected CAN registers 34
Table 7-1 Can_30_Flexcan4_InitMemory 39
Table 7-2 Can_30_Flexcan4_Init 40
Table 7-3 Can_30_Flexcan4_ChangeBaudrate 41
Table 7-4 Can_30_Flexcan4_CheckBaudrate 41
Table 7-5 Can_30_Flexcan4_SetBaudrate 42
Table 7-6 Can_30_Flexcan4_GetVersionInfo 42
Table 7-7 Can_30_Flexcan4_GetStatus 43
Table 7-8 Can_30_Flexcan4_ControllerStatePtrType 44
Table 7-9 Can_30_Flexcan4_GetControllerErrorState 44
Table 7-10 Can_30_Flexcan4_GetControllerTxErrorCounter 45
Table 7-11 Can_30_Flexcan4_GetControllerRxErrorCounter 45
Table 7-12 Can_30_Flexcan4_SetControllerMode 46
Table 7-13 Can_30_Flexcan4_Write 46
Table 7-14 Can_30_Flexcan4_CancelTx 47
Table 7-15 Can_30_Flexcan4_SetMirrorMode 47
Table 7-16 Can_30_Flexcan4_SetSilentMode 48
Table 7-17 Can_30_Flexcan4_CheckWakeup 49
Table 7-18 Can_30_Flexcan4_CheckWakeup 49
Table 7-19 Can_30_Flexcan4_DisableControllerInterrupts 50
Table 7-20 Can_30_Flexcan4_EnableControllerInterrupts 50

Table 7-21 Can_30_Flexcan4_MainFunction_Write 51
Table 7-22 Can_30_Flexcan4_MainFunction_Read 51
Table 7-23 Can_30_Flexcan4_MainFunction_BusOff 52
Table 7-24 Can_30_Flexcan4_MainFunction_Wakeup 52
Table 7-25 Can_30_Flexcan4_MainFunction_Mode 53
Table 7-26 Can_30_Flexcan4_RamCheckExecute 53
Table 7-27 Can_30_Flexcan4_RamCheckEnableMailbox 54
Table 7-28 Can_30_Flexcan4_RamCheckEnableController 54
Table 7-29 Appl_30_Flexcan4_GenericPrecopy 55
Table 7-30 Appl_30_Flexcan4_GenericConfirmation 56
Table 7-31 Appl_30_Flexcan4_GenericConfirmation 56
Table 7-32 Appl_30_Flexcan4_GenericPreTransmit 57
Table 7-33 Appl_30_Flexcan4CanTimerStart 57
Table 7-34 Appl_30_Flexcan4CanTimerLoop 58
Table 7-35 Appl_30_Flexcan4CanTimerEnd 59
Table 7-36 Appl_30_Flexcan4CanInterruptDisable 59
Table 7-37 Appl_30_Flexcan4CanInterruptRestore 60
Table 7-38 Appl_30_Flexcan4_CanOverrun 61
Table 7-39 Appl_30_Flexcan4_CanFullCanOverrun 61
Table 7-40 Appl_30_Flexcan4_CanCorruptMailbox 62
Table 7-41 Appl_30_Flexcan4_CanRamCheckFailed 62
Table 7-42 Appl_30_Flexcan4CanWriteProtectedRegister16 63
Table 7-43 Appl_30_Flexcan4CanWriteProtectedRegister32 64
Table 7-44 Appl_30_Flexcan4CanReadProtectedRegister16 64
Table 7-45 Appl_30_Flexcan4CanReadProtectedRegister32 65
Table 7-46 Appl_30_Flexcan4CanPowerOnGetBaseAddress 66
Table 7-47 Services used by the CAN 67
Table 9-1 Glossary 71
Table 9-2 Abbreviations 71

2 Hardware Overview

The following table summarizes information about the CAN Driver. It gives you detailed information about the derivatives and compilers. As very important information the documentations of the hardware manufacturers are listed. The CAN Driver is based upon these documents in the given version.

Derivative	Compiler	Hardware Manufacturer Document	Version
S32G27XX	GreenHills IAR GNU DIAB ARM6 LLVM DIAB	S32G Reference Manual	Rev.1, Nov 2019
S32K2TV		S32K2XX Reference Manual	Rev. 1 Draft N, 07/2019
S32K3XX		S32K3XX Reference Manual	Rev. 1 Draft E, 02/2020
S32R4XX		S32R4XX Reference Manual	Rev. 3 Draft C, 05/2021
KW45		KW45B41Z Reference Manual	Rev. 2, 04/28/2021
SAF85XX		SAF85xx Digital Reference Manual	Rev 1.8 01/28/2022
MWCT2xx		S32K3xx Reference Manual (Comparison list to S32K3xx)	Rev. 4, 04/2022
S32K341 S32K3x8		S32K3xx Reference Manual	Rev. 4, 04/2022
S32K38x S32K39x		S32K396 Reference Manual	Rev. 1 Draft H, 12/2021
S32G3xx		S32G3 Reference Manual	Rev. 1, 11/2021
NCJ29D6		Ranger5 User Manual	Draft Rev. 0.3 06/2022

Table 2-1 Supported Hardware Overview

Derivative: This can be a single information or a list of derivatives, the CAN Driver can be used on.
Compiler: List of Compilers the CAN Driver is working with
Hardware Manufacturer Document Name: List of hardware documentation the CAN Driver is based on.
Version: To be able to reference to this hardware documentation its version is very important.

3 Hardware Software Interface (HSI)

The CAN driver is based upon the hardware manufacturer documents given in Table 2-1 and is applicable for the FlexCAN. A detailed list of the used registers can be found in the code implantation.

A detailed description on how to access the FlexCAN controller is given in section 5.9 ‘Hardware Specific’.

4 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module CAN as specified in [1].

Since each hardware platform has its own behavior based on the CAN specifications, the main goal of the CAN Driver is to give a standardized interface to support communication over the CAN bus for each platform in the same way. The CAN Driver works closely together with the higher layer CAN Interface.

Supported AUTOSAR Release*:	4	
Supported Configuration Variants: **(Supported AUTOSAR Standard Conform Features)**	Pre-Compile Post-Build Loadable Post-Build Selectable (MICROSAR Classic Identity Manager)	
Vendor ID:	CAN_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	CAN_MODULE_ID	80 decimal (according to ref. [2])
AR Version:	CAN_AR_RELEASE_MAJOR_VERSION CAN_AR_RELEASE_MINOR_VERSION CAN_AR_RELEASE_REVISION_VERSION	AUTOSAR Release version (BCD coded)
SW Version:	CAN_SW_MAJOR_VERSION CAN_SW_MINOR_VERSION CAN_SW_PATCH_VERSION	MICROSAR Classic Can module version (BCD coded)

* For the precise AUTOSAR Release 4.x please see the release specific documentation.

4.1 Architecture Overview

The following figure shows where the CAN is located in the AUTOSAR architecture.

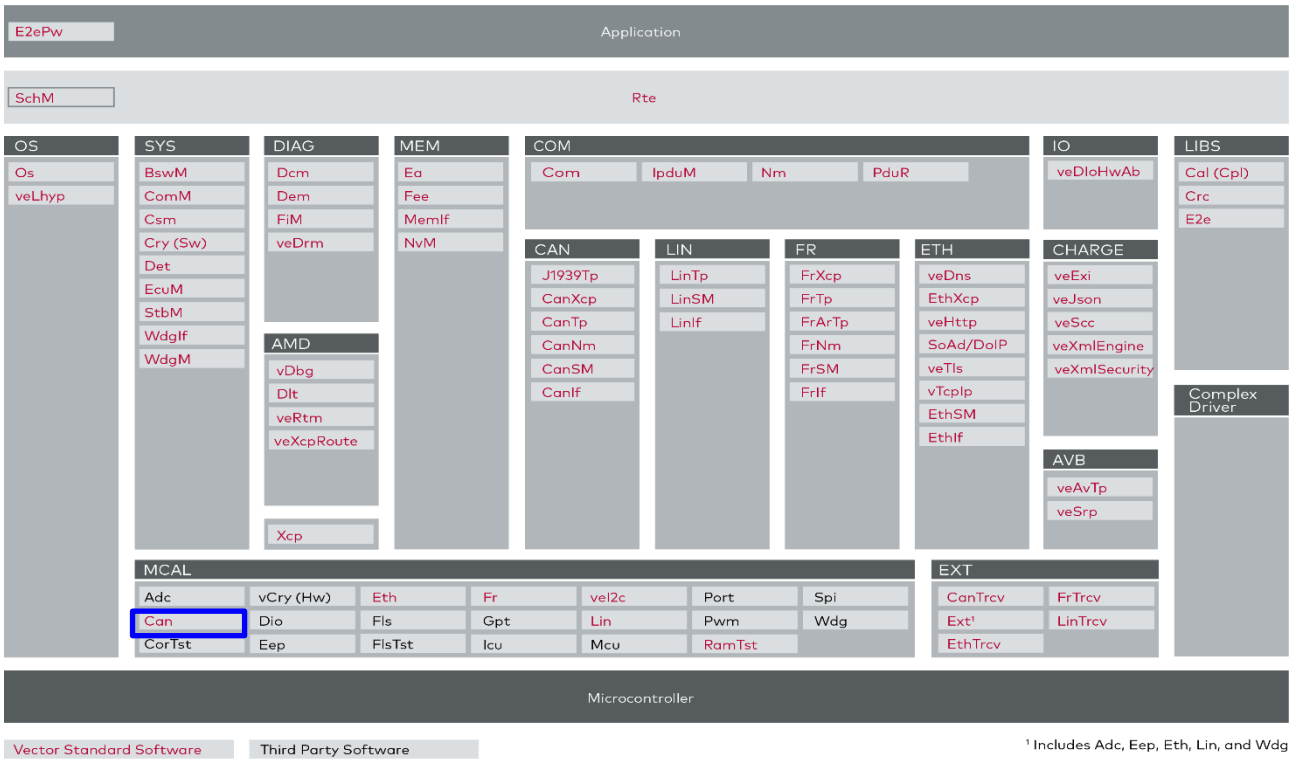


Figure 4-1 AUTOSAR architecture

The next figure shows the interfaces to adjacent modules of the CAN. These interfaces are described in chapter 7.

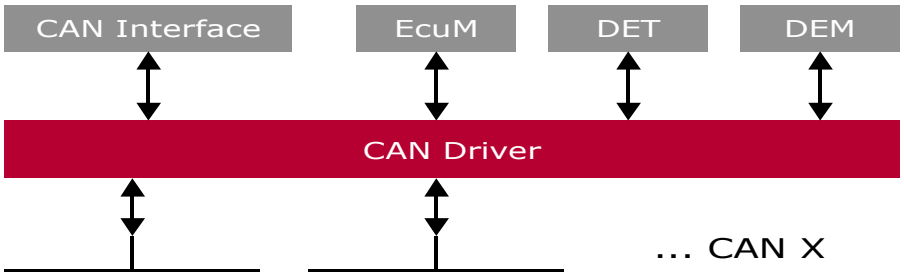


Figure 4-2 Interfaces to adjacent modules of the CAN

5 Functional Description

5.1 Features

The features listed in this chapter cover the complete functionality specified in [1].

The "supported" and "not supported" features are presented in the following table.

Feature Naming	Short Description	supported
Initialization		
PowerOnInit	General driver initialization function Can_Init()	■
DeInit	Support uninitialization routine (since ASR 4.4.0)	□
InitMemory	Support initialization of memory at power on.	■
Communication		
Transmission	Transmitting CAN frames.	■
Transmit confirmation	Callback for successful Transmission.	■
Reception	Receiving CAN frames.	■
Receive indication	Callback for receiving frame.	■
Controller Modes		
Sleep	Controller support `SLEEP` mode (power saving).	□
Wakeup over CAN	Controller support `WAKEUP` over CAN. Limitation: Can_CheckWakeup() is not encapsulated by preprocessor switch (since ASR 4.2.0)	□
Stop	Controller support `STOP` mode (passive to CAN bus).	■
Bus Off detection	Callback for `BUSOFF` event.	■
Silent Mode	Support Silent Mode where the controller only listen passive.	■ (see 5.4.5)
Wakeup over ICU	Support wakeup over ICU (since ASR 4.2.0).	□
MirrorMode / BusMirroring	Support message mirroring where the controller supports generic confirmation function for mirroring and support an API to activate and deactivate this.	■
Polling Modes		
Tx Confirmation	Support polling mode for Transmit confirmation.	■
Rx Reception	Support polling mode for Reception.	■
Wakeup	Support polling mode for `WAKEUP` event.	□
Bus Off	Support polling mode for `BUSOFF` event.	■
Mode	Support polling mode for mode transition.	■
Mailbox objects		
Tx BasicCAN	Standard mailbox to send CAN frames (Used by CAN Interface data queue).	■
Tx Hardware FIFO	Using a FIFO mechanism for Tx BasicCAN mailbox	□

Multiplexed Tx	Using 3 mailboxes for Tx BasicCAN mailbox (external priority inversion avoided).	■
Tx FullCAN	Separate mailbox for special Tx message used.	■
Maximum amount	Available amount of mailboxes.	(see 5.3.1)
Rx FullCAN	Separate mailbox for special Rx message used.	■
Maximum amount	Available amount of mailboxes.	(see 5.3.1)
Rx BasicCAN	Standard mailbox to receive CAN frame (depending on hardware, FIFO or shadow buffer supported).	■
Maximum amount	Available amount of BasicCAN objects.	(see 5.3.1)
CanHardwareObject SymbolicName values	Limitation: “Symbolic Name Values” may change their values after precompile phase so do not use it for Link-time or Post-build variants. (since ASR 4.0.0)	■
Others		
DET	Support Development Error Detection (error notification).	■
Version API	API to read out component version.	■
Maximum supported controllers	Maximum amount of supported controllers (hardware channels).	8
Cancellation of Tx objects	Support of Tx Cancellation (out of hardware). Avoid internal priority inversion.	■
Identical ID cancellation	Tx Cancellation also for identical IDs.	■
Standard ID types	Standard Identifier supported (Tx and Rx).	■
Extended ID types	Extended Identifier supported (Tx and Rx).	■
Mixed ID types	Standard and Extended Identifier supported (Tx and Rx).	■
CAN FD Mode1	FD frames with baudrate switch (BRS; Tx and Rx).	■
CAN FD Mode2	FD frames up to 64 data bytes (FULL support; Tx and Rx).	■
Hardware Loop Check (Timeout monitoring)	To avoid possible endless loops (occur by hardware issue).	■
Multiple CAN driver	API infixed CAN driver support	■
Pretended Networking	Support pretended networking (since ASR 4.2.0)	□
Individual Polling	Support individual polling mode (selectable for each mailbox separate). Limitation: AutoSar feature multiple cyclic functions with different time periods are not supported. The polling take part in the same main function calls (since ASR 4.2.0).	■*
TriggerTransmit	Support trigger transmit (since ASR 4.2.0)	□
EcuC partition map	Support EcuC partition mapping (since ASR 4.4.0).	□
AutoSar extensions		
Multiple Rx Basic CAN	Support Multiple Rx BasicCAN objects. This gives the possibility to use multiple filters and optimize acceptance Filtering as well as avoid `OVERRUN`.	□* (see 5.3.1)

Multiple Tx Basic CAN	Support Multiple Tx BasicCAN objects. Used to send different Tx groups over separate mailboxes with different buffering behavior (see CanInterface). (“Cancellation of Tx objects” is not possible with this feature activated)	■*
Rx Queue	Support Rx Queue. This give the possibility to buffer received data in interrupt context but handle it asynchronous in polling task.	■*
Hardware Loop Check by application	“Hardware Loop Check” can be defined to be done by application (special API available)	■
Configurable “Nested CAN interrupts”	Nested CAN interrupts allowed and can be also switched to none-nested. Nested means that higher prior interrupts may appear in the already active CAN interrupt context.	□
Support Mixed ID	Force CAN Driver to handle mixed ID (standard and extended ID) at pre-compile-time to expand the ID type later on.	■
Optimize for one controller	Activate this for 1 controller systems when you never will expand to multi-controller. So that the CAN Driver works more efficient	■
Size of Hw HandleType	Support 8bit or 16bit Hardware Handles depend on hardware usage.	■
Generic PreCopy	Support a callback function for receiving any CAN message (following callbacks could be suppressed) Limitation: AutoSar feature name is LPDU callback function this is not fully supported because the API signature is different (since ASR 4.2.0)	■
Generic Confirmation	Support a callback function for successful transmission of any CAN message (following callbacks could be suppressed)	■
Get Hardware Status	Support a API to get hardware status information (see Can_GetStatus(), Can_GetControllerStatus(), Can_GetControllerMode(), CanGetControllerRxErrorCounter(), CanGetControllerTxErrorCounter())	■
Interrupt Category selection	Support Category 1 or Category 2 Interrupt Service Routines for OS	■
Overrun Notification	Support DET or application notification cause by `OVERRUN` (`OVERWRITE`) of an Rx message (BasicCAN and FullCAN) Limitation: overrun will be only notified when DET is activated (since ASR 4.4.0 the overrun will be also notified without DET)	■ (see
RAM Check	Support CAN mailbox RAM check	■ (see
Extended RAM Check	Support extended RAM check. Handling of individual deactivated mailboxes and controllers.	■**
Generic PreTransmit	Support a callback function with pointer to Data, right before this data will be written in hardware mailbox buffer to send. (Use this to change data right before transmission)	■

Table 5-1 Supported features

■ Feature is supported

☐ Feature is not supported

* HighEnd Licence only

** Project specific (may not be available)

5.2 Initialization

`Can_Init()` has to be called to initialize the CAN Driver at power on and initialize the CAN controllers.

After this call the controller stays in `STOP` mode until the CAN Interface changes to `START`.

`Can_InitMemory()` is an additional service function to reinitialize the memory to bring the driver back to a pre-power-on state (not initialized). Afterwards `Can_Init()` has to be called again. It is recommended to use this function before calling `Can_Init()` to secure that no startup-code specific pre-initialized variables affect the driver startup behavior.

5.3 Communication

`Can_Write()` is used to send a message over the mailbox object given as "Hth". The data, DLC and ID is copied into the hardware mailbox object and a send request is set. After sending the message the CAN Interface `CanIf_TxConfirmation()` function is called. Right before the data is copied in mailbox buffer the ID, DLC and data may be changed by `Appl_GenericPreTransmit()` callback.

When “Generic Confirmation“ is activated the callback `Appl_GenericConfirmation`() will be called before `CanIf_TxConfirmation()` and the call to this can be suppressed by `Appl_GenericConfirmation`() return value.

For Tx messages the ID will be copied. (Exception: feature “Dynamic FullCAN Tx ID” is deactivated, then the FullCAN Tx messages will be only set while initialization)

If the mailbox is currently sending the status busy will be returned. Then the message may be queued in the CAN Interface (if feature is active).

If cancellation in hardware is supported the lowest priority ID inside currently sending object is canceled, and therefore re-queued in the CAN Interface.

`Appl_GenericPreCopy`() (if activated) is called and depend on return value also `CanIf_RxIndication()` as a CAN Interface callback, is called when a message is received. The receive information like ID, DLC and data are given as parameter.

When Rx Queue is activated the received messages (polling or interrupt context) will be queued (same queue over all channels). The Rx Queue will be read by calling `Can_Mainfunction_Read` () and the Rx Indication (like `CanIf_RxIndication`()) will be called out of this context. Rx Queue is used for Interrupt systems to keep interrupt latency time short.

5.3.1 Mailbox Layout

The FlexCAN4 Controller makes use of the Enhanced RxFifo capable of storing up to 64 data bytes. For CAN-FD Mode 2 the number of available mailboxes depends on the maximum number of data bytes configured (please refer to the manufacturer's Reference Manual.

The generation tool supports a flexible allocation of message buffers. In the following tables the possible mailbox layout is shown (the range for each mailbox types depend on the used mailboxes).



Please note
The Rx BasicCAN object is represented by the Enhanced RxFifo of the CAN controller. The feature "Multiple BasicCAN" is not supported.

Object Number*)	Object Type	Amount of Objects	Comment
0 *)	Rx BasicCAN	1	For the BasicCAN object the Enhanced RxFifo is used. All CAN messages, except for the FullCAN messages, are received via the BasicCAN message object.
0 … a	Rx FullCAN	0 … n_{max}	These objects are used to receive specific CAN messages. The user defines statically (Generation Tool) that a CAN message should be received in a FullCAN message object. The Generation Tool distributes the message to the FullCAN objects.
a+1 … b	Tx BasicCAN	X * Y	These objects are used to send several messages. If the transmit message object is busy, the transmit request is stored in a CAN Interface queue (if activated). X = amount of TxBasicCANs, Y = 1 or 3 depend on Multiplexed Tx feature.
b+1 … c	Tx FullCAN	0 ... n_{max}	These objects are used to send a certain message. The user must define statically (Generation Tool) which CAN messages are located in such Tx FullCAN objects. The Generation Tool distributes the messages to the FullCAN objects.
c+1 … d	Unused	0 ... n_{max}	These objects are not used. It depends on the configuration of receive and transmit objects if unused objects are available.

Table 5-2 Hardware mailbox layout

Please note
n_{max} is the maximum amount of available mailboxes per CAN Controller and depends on the used derivative.

*) Rx BasicCAN is located in the Enhanced RxFifo memory section and has object number 0 assigned (in this memory section). Rx FullCAN, Tx BasicCAN and Tx FullCAN objects make use of the hardware mailboxes beginning with object number 0.

The “CanObjectId” (ECUc parameter) numbering is done in following order: FullCAN Tx, BasicCAN Tx, Unused, FullCAN Rx, BasicCAN Rx (like shown above). “CanObjectId’s” for next controller begin at end of last controller. Gaps in “CanObjectId” for unused mailboxes may occur.

5.3.2 Mailbox Processing Order

The hardware mailbox will be processed in following order:

Object Type	Order / priority to send or receive
Tx FullCAN	CAN ID Low to High
Tx BasicCAN	CAN ID Low to High
Rx FullCAN	Object ID Low to High
Rx BasicCAN	FIFO

Please note
Rx and Tx Polling Mode:

FullCANs will be processed before BasicCANs.

Tx Interrupt Mode:

BasicCANs will be processed before FullCANs.

Rx Interrupt Mode:

BasicCANs will be processed before FullCANs.

The order between Rx and Tx mailboxes depends on the call order of the polling tasks or the interrupt context and cannot be guaranteed.

The Rx Queue will work like a FIFO filled with the method mentioned above.

5.3.3 Acceptance Filter for BasicCAN

The Enhanced RxFifo makes use of the Enhanced Rx Filter Table with a code code/mask scheme. The Enhanced Rx Filter Table can keep up to 128 standard filters or up to 64 extended filters.

For configuration details refer to the manufacturer's Reference Manual, please.

If no message should be received set the filter should be set to "close".

5.3.4 Remote Frames

Remote Frames rejected in Hardware	Remote Frames rejected in Software
X	

The CAN driver initializes the CAN controller to NOT receive remote frames. Therefor no additional action is required during runtime by the CAN driver for remote frame filtering. Remote frames will not have any influence on communication.

5.4 States / Modes

You can change the CAN Cell mode via Can_SetControllerMode(). The last requested transition will be executed. The upper layer should take care about valid transitions.

Following mode changes are supported:

`CAN_T_START`

`CAN_T_STOP`

`CAN_T_SLEEP`

`CAN_T_WAKEUP`

Notification of mode change may occur asynchronous by notification `CanIf_ControllerModeIndication()`

5.4.1 Start Mode (Normal Running Mode)

This is the mode where communication is possible. This mode has to be set after initialization because the controller is first in `STOP`. See section 5.8.2 for hardware loops that may occur during start mode transition.

5.4.2 Stop Mode

If stop mode is requested then the CAN module is switched into Freeze mode. The CAN Controller is waiting for ongoing reception and transmission processes to be finished until the Freeze mode is entered successfully. This mode transition is monitored by a particular Hardware Loop Check. See section 5.8.2 for hardware loops that may occur during stop mode transition. In stop mode no transmission or reception is possible and incoming message frames are not acknowledged. All pending transmissions, receive and status indications are cancelled.

5.4.3 Sleep Mode

Only an emulated sleep is supported.

`EcuM_CheckWakeup()` is called in case a `WAKEUP` over CAN event occurred.

`Can_Cbk_CheckWakeup()` can be called to check `WAKEUP` state.

`CAN_T_WAKEUP` has to be set with `Can_SetControllerMode()` to leave this mode.

If wakeup support is deactivated by configuration, the controller goes into `STOP` mode instead of `SLEEP`.

5.4.4 Bus Off

`CanIf_ControllerBusOff()` is called when the controller detects a `BUSOFF` event. The mode is automatically changed to `STOP`. The upper layers have to care about returning to normal running mode by calling `START`.



Please note:
After a Bus Off event occurs, the CAN Controller is waiting for 128x11 consecutive recessive bits on the bus for recovery (ISO compliant recovery time). The Bus Off recovery is performed asynchronously in start mode transition.
If a transition to STOP is triggered during Bus Off state the CAN controller leaves the Bus Off condition and with the next transition to START it will wait for 11 consecutive recessive bits.

5.4.5 Silent Mode

Support API (`Can_SetSilentMode()`) to switch into ‘Silent Mode’ where the controller does not take part on CAN bus communication (no ACK) but can listen for messages. Refer to ISO 11898 bus monitoring.
Change in ‘Silent Mode’ and back again to normal operating mode is only possible in `STOP`. To get RX notifications in ‘Silent Mode’ it is necessary to switch to `START` mode.
In ‘Silent Mode’ it is possible to change the baud rate and switch to `START` mode, used to scan baud rates by detect reception.



Caution
Don’t use any other API than Can_SetSilentMode(x, CAN_SILENT_INACTIVE) inside activated “Silent Mode” except Can_SetControllerMode(`START` or `STOP`) and Can_ChangeBaudrate() and Can_SetBaudrate().

If you use CAN FD baud rates to be scanned, use a FD message for detection.

The FlexCAN describes this mode as Listen-Only Mode:

After Entering Listen-Only Mode the controller operates in Error Passive mode. The error counters are frozen and will not be updated. So, the error counters cannot be used to detect a wrong baudrate.

In Listen-Only Mode the controller does not take part in the bus communication. In this mode transmission is disabled and the controller is recessive on the bus.

Only messages acknowledged by another node in the network will be received. There is no self ACK or internal rerouting. This means there must be nodes in the network that are able to communicate with each other with the correct baudrate set. Receiving such a message indicates that the correct baudrate is found.



Limitations
Only messages acknowledged by another node in the network will be received (there are already nodes in the network that are able to communicate with each other with the correct baudrate set).

Error counters cannot be used to detect a wrong baudrate.

5.5 Re-Initialization

A call to `Can_ChangeBaudrate() or Can_SetBaudrate()` cause a re-initialization of a dedicated CAN Controller. Pending messages may be processed before the transition will be finished. A re-initialization is only possible out of `STOP` and does not change to another mode.

After re-initialization all CAN communication relevant registers are set to initial conditions.

5.6 CAN Interrupt Locking

`Can_DisableControllerInterrupts()` and `Can_EnableControllerInterrupts()` are used to disable and enable the controller specific Interrupt, Rx, Tx, `WAKEUP` and `BUSOFF` (/ Status) together. These functions can be called nested.

5.7 Main Functions

`Can_MainFunction_Write()`, `Can_MainFunction_Read()`, `Can_MainFunction_BusOff()` and `Can_MainFunction_Wakeup()` called by upper layer to poll the events if the specific polling mode is activated. Otherwise these functions return without action and the events will be handled in interrupt context.

When individual polling is activated only mailboxes that are configured as to be polled will be polled in the main functions “`Can_MainFunction_Write()`” and “`Can_MainFunction_Read()`” all others handled in interrupt context.

When Rx Queue is activated the queue is filled in interrupt or polling context like configured. But the processing (indications) will be done in “`Can_MainFunction_Read()`” context.

Can_MainFunction_Mode() called by upper layer to poll asynchronous mode transition notifications.

5.8 Error Handling

5.8.1 Development Error Reporting

Development errors are reported to DET using the service `Det_ReportError()`, if the pre-compile parameter CAN_DEV_ERROR_DETECT == STD_ON.

The tables below, shows the API ID and Error ID given as parameter for calling the DET.

Instance ID is always 0 because no multiple instances are supported.

Errors reported to DET:	
Error ID	**Short Description**
CAN_E_PARAM_POINTER	API gets an illegal pointer as parameter.
CAN_E_PARAM_HANDLE	API get an illegal handle as parameter
CAN_E_PARAM_DLC	API get an illegal DLC as parameter
CAN_E_PARAM_CONTROLLER	API get an illegal controller as parameter
CAN_E_UNINIT	Driver API is used but not initialized
CAN_E_TRANSITION	Transition for mode change is illegal
CAN_E_DATALOST (value: 0x07, AutoSar extension)	Rx `OVERRUN` (`OVERWRITE`) detected
CAN_E_PARAM_BAUDRATE (value: 0x08, AutoSar extension)	Selected baudrate is not valid
CAN_E_RXQUEUE (value: 0x10, AutoSar extension)	Rx Queue `OVERRUN` (last received message is lost, and will not be received → increase queue size)
CAN_E_TIMEOUT_DET (value: 0x11, AutoSar extension)	Hardware Loop timeout detected.

Table 5-3 Errors reported to DET

API from which the errors reported to DET:	
API ID	**Functions using that ID**
CAN_VERSION_ID	Can_GetVersionInfo()
CAN_INIT_ID	Can_Init()
CAN_INITCTR_ID	CanHL_ReInit()
CAN_SETCTR_ID	Can_SetControllerMode()
CAN_DIINT_ID	Can_DisableControllerInterrupts()
CAN_ENINT_ID	Can_EnableControllerInterrupts()
CAN_WRITE_ID	Can_Write(), Can_CancelTx()
CAN_TXCNF_ID	CanHL_TxConfirmation()
CAN_RXINDI_ID	CanBasicCanMsgReceived(), CanFullCanMsgReceived()
CAN_CTRBUSOFF_ID	CanHL_ErrorHandling()

CAN_CKWAKEUP_ID	CanHL_WakeUpHandling(), Can_Cbk_CheckWakeup()
CAN_MAINFCT_WRITE_ID	Can_MainFunction_Write()
CAN_MAINFCT_READ_ID	Can_MainFunction_Read()
CAN_MAINFCT_BO_ID	Can_MainFunction_BusOff()
CAN_MAINFCT_WU_ID	Can_MainFunction_Wakeup()
CAN_MAINFCT_MODE_ID	Can_MainFunction_Mode()
CAN_CHANGE_BR_ID	Can_ChangeBaudrate()
CAN_CHECK_BR_ID	Can_CheckBaudrate()
CAN_SET_BR_ID	Can_SetBaudrate()
CAN_GETCTR_STATE_ID	Can_GetControllerErrorState()
CAN_GETCTR_MODE_ID	Can_GetControllerMode()
CAN_GETCTR_RXCNT_ID	Can_GetControllerRxErrorCounter()
CAN _GETCTR_TXCNT_ID	Can_GetControllerTxErrorCounter()
CAN_HW_ACCESS_ID (value: 0x60, AUTOSAR extension)	ApplCanTimerLoop() Can_GetStatus() Can_SetMirrorMode() Can_SetSilentMode() Can_RamCheckExecute() Can_RamCheckEnableMailbox() Can_RamCheckEnableController()

Table 5-4 API from which the Errors are reported

5.8.1.1 Parameter Checking

AUTOSAR requires that API functions check the validity of their parameters (Refer to [1]). These checks are for development error reporting and can be enabled and disabled separately. Refer to the configuration chapter where the enabling/disabling of the checks is described. Enabling/disabling of single checks is an addition to the AUTOSAR standard which requires enable/disable the complete parameter checking via the parameter `CAN_DEV_ERROR_DETECT`.

5.8.1.2 Overrun/Overwrite Notification

As AUTOSAR extension the `OVERRUN` detection may be activated by configuration tool. The notification can be configured to call an DET or application call `Appl_CanOverrun`() or `Appl_CanFullCanOverrun`().

Object Type	Notification	Comment
Rx BasicCAN	`OVERRUN`	Unconditionally
Rx FullCAN	`OVERWRITE`	If BasicCAN filter is closed to the ID.
Rx FullCAN	`UNDETECTABLE`	If BasicCAN filter is open to the ID. New, none processed messages will be received in BasicCAN.

Object Type	Notification	Comment
		Undefined behavior, upper Layer may discard the message.

Table 5-5 Overrun/Overwrite state overview

To ensure that every FullCAN `OVERWRITE` issues a notification, the BasicCAN filter must be configured that none of the FullCAN messages can pass it. Otherwise only an `OVERRUN` will occur, that the CAN driver cannot notify.

5.8.2 Hardware Loop Check / Timeout Monitoring

The feature “Hardware Loop Check” is used to break endless loops caused by software or hardware issue. This feature is configurable see Chapter 7 and also Timeout Duration description.

Synchronous part of mode transitions will be also limited by this timeout mechanism, which is no issue but a timing limit. (following asynchronous part of mode transition is handled without HardwareLoopCheck)

The Hardware Loop Check will be handled by CAN driver internal except when setting “Hardware Loop Check by Application” is activated.

Nevertheless, refer to “short description” below there may be activities that should be initiated by the application (e.g. reset of CAN Controller or special mode transitions). In this case the “Hardware Loop Check by Application” is recommended to be used to handle the explicit loop like described.

5.8.2.1 Critical Loops

A loop exception has to be handled by application like described below.

Loop Name / source	Short Description
kCanLoopLeaveDisableModeInit	This loop is used to monitor the mode transition to leave Disable mode. - Call context: CanHL-Reinit () - Execution time: until LPM_ACK bit is reset. For conditions refer to the Reference Manual. - In case of failure the application needs to restart with power on initialization.

Loop Name / source	Short Description
kCanLoopEnterDisableModeInit	This loop is used to monitor the mode transition to Disable mode. - Call context: CanHL-Reinit () - Execution time: until LPM_ACK bit is set. For conditions refer to the Reference Manual. - In case of failure the application needs to restart with power on initialization.
kCanLoopEnterFreezeModeInit	This loop is used to monitor the mode transition to Freeze mode. - Call context: CanHL-Reinit () - Execution time: until NOT_RDY and FRZ_ACK bits are set. For conditions refer to the Reference Manual. - Hardware condition for mode transition: “Waits to be in either intermission, passive error, bus off or idle state” (citation from hardware manual of semiconductor manufacturer). Consequential a dominant level at the Rx pin caused by the transceiver or other external influences may lead to longer blocking periods. - In case of failure the application needs to restart with power on initialization.
kCanLoopResetInit	This loop is used in function CanHL-Reinit () when the CAN driver requests the soft reset of the CAN controller. - After soft reset is requested the CAN controller resets its internal state machines and resets the following registers: CANx_MCR (except the MDIS bit), CANx_ESR, CANx_IMASKx, CANx_IFLAGx. - Configuration registers that control the interface to the CAN bus are not affected by soft reset. The following registers are unaffected: CANx_CTRL, CANx_RXGMASK, CANx_RX14MASK, CANx_RX15MASK and all message buffers. - Soft reset is synchronous and has to follow a request/acknowledge procedure across clock domains, So, it may take some time to fully propagate its effect. - The SOFTRST bit remains asserted while reset is pending and it is automatically negated when reset completes. - In case of failure the application needs to restart with power on initialization.

Table 5-6 Critical Loops

5.8.2.2 Uncritical Loops

No additional application handling needed after loop break.

kCanLoopMsgReception	It is used to control message reception: - After a message frame has been received into the serial message buffer SMB the CAN hardware transfers its data from the SMB to the matching receive buffer (FullCAN). During this transfer the CAN message buffer is set busy. The CAN driver must wait until the message buffer is no longer busy to process the message buffer content. - Loop index is used in the functions CanFullCanMsgReceived() to wait until the message is completely received in the message buffer.

Table 5-7 Uncritical Loops

5.8.2.3 Loops used for synchronous mode transitions

Driver handle the mode transition in asynchronous way afterwards, so no additional handling is necessary.

Loop Name / source	Short Description
kCanLoopMode	Used for short time mode transition blocking (short synchronous timeout). Use for mode changes START, STOP, SLEEP and WAKEUP. No issue when timeout occurs.

Table 5-8 Hardware Loop Check

5.8.3 CAN RAM Check

The CAN Driver supports a check of the CAN Controller's mailbox data registers (ID, DLC, data). The CAN Controller RAM check is called internally every time a power on is executed within Initialization, or a CAN bus `WAKEUP` event happen. The CAN Driver verifies that no used mailboxes are corrupt. A mailbox is considered corrupt if a predefined pattern is written to the appropriate mailbox registers and the read operation does not return the expected pattern. If a corrupt mailbox is found the function `Appl_CanCorruptMailbox()` is called. This function tells the application which mailbox is corrupt.

After the check of all mailboxes the CAN Driver calls the call back function `Appl_CanRamCheckFailed()` if at least one corrupt mailbox was found. The application must decide if the CAN Driver disables communication or not by means of the call back function's return value. If the application has decided to disable the communication there is no possibility to enable the communication again until the next call to Can_Init() or a `WAKEUP` event occurs.

The CAN RAM check functionality itself can be activated via Generation Tool.

5.8.4 Extended RAM Check

The CAN Driver supports a check for all accessible CAN Controller's control registers and mailbox registers. The extended RAM check will be executed while power on initialization and by direct call. Mailboxes will be deactivated when pattern check or configured values are corrupt. The CAN Controller will be deactivated when one mailbox is corrupt or the controller's registers failed the pattern check or configured values are corrupt. Mailbox and controller stay deactivated until they are explicit activated again.

API to execute extended RAM check:

`Can_RamCheckExecute()`

Callouts to notify corrupt mailboxes or controllers:

`CanIf_RamCheckCorruptController(), CanIf_RamCheckCorruptMailbox()`

API to activate the mailbox or controller again:

`Can_RamCheckEnableMailbox(),Can_RamCheckEnableController()`

(See also API description)

The CAN RAM check functionality itself can be activated via Generation Tool.

5.8.5 Security Event Reporting

The CAN driver supports the detection and reporting of security events observed by the CAN controller. The reporting will be executed during the driver error handling, either in interrupt, or polling context. The associated context data will be reported to the CanIf when a security event is detected.

The security event reporting feature can be activated through the parameter CanEnableSecurityEventReporting in the generation tool.

Three types of error reporting are provided with the security event reporting feature:

Error state passive

Reported when the CAN driver detects a transition to error state passive. This is reported when calling the API `CanHL_30_Flexcan4_ControllerErrorStatePassive()`, which forwards the associated context data to the CanIf API `CanIf_ControllerErrorStatePassive()` during the error handling.

Error state bus off

Reported when the CAN driver detects a bus off event. The driver calls the CanIf API `CanIf_30_Flexcan4ControllerBusOff` during the error handling when a bus off event is detected.

Error reporting (error types)

Specific CAN error types are reported to the the `CanIf API CanIf_ErrorNotification()` from the CAN driver API `CanHL_30_Flexcan4_ErrorNotification()` during the error handling.

The supported CAN error types are listed in Table 5-9:

Security Event	Short Description
Error bit monitoring 1	Reported when a “0” was transmitted and a “1” was read back
Error bit monitoring 0	Reported when a “1” was transmitted and a “0” was read back
Error acknowledge	Reported when acknowledgement check failed
Error form	Reported when a violation of the frame format is detected
Error stuffing	Reported when the stuffing bits is not as expected
Error CRC	Reported when the CRC check failed

Table 5-9 Security Events

5.8.5.1 FlexCAN Security Event Reporting Restrictions

Error reporting (error types)

Reading the FlexCAN’s error status register clears all error events listed in Table 5-9 Security Events.

Besides the Security Event Reporting the API functions Can_30_Flexcan4_GetStatus() and Can_30_Flexcan4_GetControllerErrorState() read the error status register as well.

Therefore, using these functions in combination with the Security Event Reporting may lead to race condition so that errors to be reported by Security Event Reporting are lost or vice versa.



Caution
Using Security Event Reporting together with Can_30_Flexcan4_GetStatus() or Can_30_Flexcan4_GetControllerErrorState () may lead to race condition so that errors are either not reported with Security event reporting or Can_30_Flexcan4_GetStatus() or Can_30_Flexcan4_GetControllerErrorState()

Error state passive

The transition to error passive is not reported by interrupt.

Therefore, in case of interrupt configuration (BusOff) the transition to passive state is only detected if one of the error events listed in Table 5-9 Security Events occurs. Furthermore if Security Event Reporting is used together with Can_30_Flexcan4_GetStatus() or Can_30_Flexcan4_GetControllerErrorState() transitions to passive state may be lost.



Caution
Using Security Event Reporting in interrupt mode error state passive transition reports may be lost because error events listed in Table 5-9 Security Events need to be reported. Furthermore there may be a loss of error state passive transitions if Security Event Reporting is used in combination with Can_30_Flexcan4_GetStatus() or Can_30_Flexcan4_GetControllerErrorState ().

5.9 Hardware Specific

5.9.1 Tx Arbitration Start Delay (TASD)

The first bit of the CRC field on the CAN bus is one of the trigger events for the CAN controller to initiate an arbitration cycle. For this case the arbitration can be delayed to optimize the performance of transmission proceedings. If an arbitration process is delayed there is a little more time left for Tx message buffer configuration but less time is reserved for the arbitration procedure.



Info
The default TASD value is 16. The default value for TASD is used if the TASD computation in the generation tool stays disabled.

5.9.2 Enhanced RxFifo

The CAN driver supports the Enhanced RxFifo feature which is available in the CAN controller. The Enhanced RxFifo is used as Rx BasicCAN object. It provides a storage capacity of 32 message frames and message frame receive order handling included. It follows that the application has more time to process message frame reception without message lost. The Enhanced RxFifo makes use of the Enhanced RxFifo Filter Table.

Legacy RxFifo is not supported.

5.9.3 Enhanced RxFifo Filter Table

The Enhanced RxFifo Filter Table can keep up to 128 Standard Filters, up to 64 Extended Filters or mixture of them, where the Extended Filters are kept in front of the Standard Filters.



Info
Only code/mask scheme is supported as filter configuration.

5.9.4 Enhanced Bit Timing

Enhanced Bit Timing is used on derivatives that supports this feature. Please refer to the manufacturer’s Technical Reference Manual.

5.9.5 Extra FD RAM

Extra FD RAM is used on derivatives that supports this feature. Please refer to the manufacturer's Technical Reference Manual.

5.9.6 Error Interrupt

The FlexCAN error interrupt source is not used by the CAN driver. Only BusOff events are handled and reported to the upper layers by the CAN driver.

5.9.7 Privileged Operating Modes

The CPU supports different operating modes (e.g. User and Supervisor mode). Depending on the derivative the R/W access to some CAN hardware registers are possibly allowed in the Supervisor mode only. Depending on the system design the CAN driver may not be allowed to switch into a privileged mode by itself. The protected registers must be accessed outside the CAN driver from an upper layer (for example handled by the Operating System) which is privileged to switch in the Supervisor mode (see chapter 6.5.1).

5.9.8 Support of S32K3xx, S32K38x, S32K9x, MWCT2xx

This driver (DrvCan_Arm32Flexcan4Asr) supports those Flexcan channels (controllers) only that have the "Enhanced RX FIFO" included.

For usage of the channels (controllers) that have the "Enhanced RX FIFO" not included but the "Legacy RX FIFO" only, the driver DrvCan_Arm32Flexcan3Asr is needed as well (both drivers are needed).

5.10 Virtual Addressing

Virtual Addressing is intended to be used in combination with operating systems that use virtual addresses instead of physical addresses to access the FlexCAN4 hardware. The driver sets the virtual base address by calling the application callback function Appl_30_Flexcan4CanPowerOnGetBaseAddress () during Can_30_Flexcan4Can_Init(). This function needs to be declared and defined by the application. It is described in section 7.2.46.



Info
To enable the Virtual Addressing feature the following code segment must be added to user configuration file:

```
#define C_ENABLE_UPDATE_BASE_ADDRESS
```

6 Integration

This chapter gives necessary information for the integration of the MICROSAR Classic CAN into an application environment of an ECU.

6.1 Scope of Delivery

The delivery of the CAN contains the files, which are described in the chapter's 6.1.1 and 6.1.2:

Dependent on library or source code delivery the marked (+) files may not be delivered.

6.1.1 Static Files

File Name	Description
Can_Local.h	This is an internal header file which should not be included outside this module
Can.c	This is the source file of the CAN. It contains the implementation of CAN module functionality.
Can.h	This is the header file of the CAN module (include API declaration)
Can_Irq.c	This is the interrupt declaration and callout file (supports interrupt configuration as link time settings)

Table 6-1 Static files

6.1.2 Dynamic Files

The dynamic files are generated by the configuration tool.

File Name	Description
Can_Cfg.h	Generated header file, contains some type, prototype and pre-compile settings
Can_Lcfg.c	Generated file contains link-time settings.
Can_PBcfg.c	Generated file contains post-build settings.
Can_DrvGeneralTypes.h	Generated file contains CAN driver part of Can_GeneralTypes.h (supported by Integrator)

Table 6-2 Generated files

6.2 Include Structure

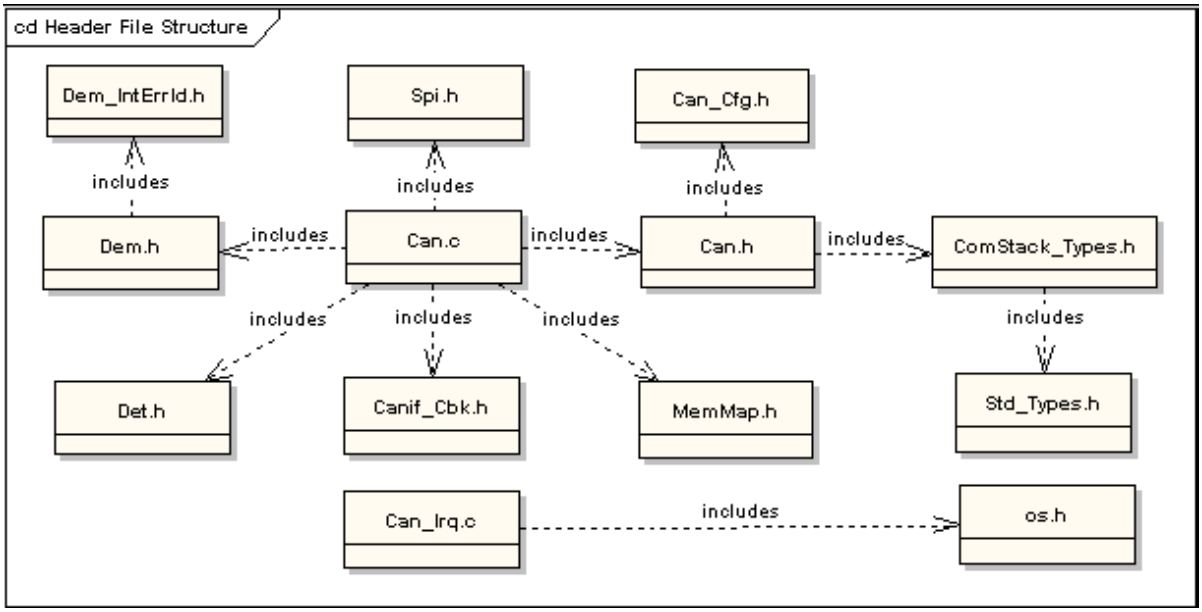


Figure 6-1 Include Structure (AUTOSAR)

Deviation from AUTOSAR Specification:

- Additionally the EcuM_Cbk.h is included by Can_Cfg.h (needed for wakeup notification API).
- ComStack_Types.h included by Can_Cfg.h, because the specified types have to be known in generated data as well.
- Os.h will be included by Can_Cfg.h because of used data types
- Spi.h is not yet used.
- Can_GeneralTypes.h will be included by Can_Cfg.h not by Can.h direct.
- Dem.h is not used

6.3 Critical Sections

The AUTOSAR standard provides with the BSW Scheduler a BSW module, which handles entering and leaving critical sections.

For more information about the BSW Scheduler please refer to [3]. When the BSW Scheduler is used the CAN Driver provides critical section codes that have to be mapped by the BSW Scheduler to following mechanism:

Critical Section Define	Description
CAN_EXCLUSIVE_AREA_0	Using inside message reception context in the functions CanFullCanMsgReceived() and (if Classic Mailbox Layout is used) CanBasicCanMsgReceived(). > Duration is short (< 30 instructions). > No API call of other BSW inside. > Disable all interrupts that serves CAN functionality.
CAN_EXCLUSIVE_AREA_1	Using inside Can_DisableControllerInterrupts() and Can_EnableControllerInterrupts() to avoid nested call of these APIs. > Duration is short (< 30 instructions) > No API call of other BSW inside. > Disable global interrupts – or – Empty in case Can_Disable/EnableControllerInterrupts() are called within context with lower or equal priority than CAN interrupt and Can_DisableControllerInterrupts(), Can_EnableControllerInterrupts() are not called nested (each other and themselves).
CAN_EXCLUSIVE_AREA_2	Using inside Can_Write() to secure software states of transmit objects. (Only called when no Vector CAN Interface is used.) > Duration is medium (< 100 instructions) > No API call of other BSW inside. > Disable global interrupts - or - Disable CAN interrupts and do not call function Can_Write() reentrant.
CAN_EXCLUSIVE_AREA_3	Using inside Tx confirmation to secure state of transmit object in case of cancellation (Only used when Vector Interface Version smaller 4.10 used). Or When Appl_GenericConfirmation() is used, lock against Can_Write() calls within (Data Consistency). > Duration is medium (< 100 instructions) > Call to CanIf_CancelTxConfirmation() inside (no more calls in CanIf). > Disable global interrupts - or - Disable CAN interrupts and do not call function Can_Write() within.
CAN_EXCLUSIVE_AREA_4	Using inside received data handling (Rx Queue treatment) to secure Rx Queue counter and data. > Duration is short (< 30 instructions) > No API call of other BSW inside. > Disable Global Interrupts - or - Disable all CAN interrupts.

CAN_EXCLUSIVE_AREA_5	Using inside wakeup handling to secure state transition. (Only in wakeup polling mode) Wakeup polling must not be interrupted by any mode change like triggered by Can_SetControllerMode(), BUSOFF-handling, Can_Init(), Can_ChangeBaudrate() or Can_SetBaudrate(). > Duration is short (< 20 instructions) > Call to DET inside. > Use CAN interrupt locks here, in case the APIs mentioned above are only called within same task level or CAN interrupt context. or No action (locks) necessary when all APIs (mentioned above) called from same context or cannot interrupt each other. or Disable global interrupts
CAN_EXCLUSIVE_AREA_6	Using inside Can_SetControllerMode() and BUSOFF to avoid nested state transition requests. Can_SetControllerMode() and BUSOFF-handling (polling or interrupt) must not be interrupted by other calls of Can_SetControllerMode(), BUSOFF-handling, Can_Init(), Can_ChangeBaudrate() or Can_SetBaudrate() > Duration is medium (< 100 instructions) > No API call of other BSW inside. > Use CAN interrupt locks here, in case the above APIs mentioned above are only called within same task level or CAN interrupt context. or No action (locks) necessary when all APIs (mentioned above) called from same context or cannot interrupt each other. or Disable global interrupts
CAN_EXCLUSIVE_AREA_7	Using inside received data handling (Tx hardware FIFO treatment) to secure Tx hardware FIFO counter and data. > Duration is short (< 30 instructions) > No API call of other BSW inside. > Disable Global Interrupts - or - Disable all CAN interrupts.

Table 6-3 Critical Section Codes

6.4 Compiler Abstraction and Memory Mapping

The objects (e.g. variables, functions, constants) are declared by compiler independent definitions – the compiler abstraction definitions. Each compiler abstraction definition is assigned to a memory section.

The following table contains the memory section names and the compiler abstraction definitions defined for the CAN Interface and illustrates their assignment among each other.

Memory Mapping Sections \ Compiler Abstraction Definitions	CAN_ CODE	CAN_STATIC_ CODE	CAN_ CONST	CAN_ CONST_PBCFG	CAN_VAR_NOINIT	CAN_ VAR_INIT	CAN_VAR_PBCFG	CAN_ INT_CTRL	CAN_ REG_CANCELL	CAN_ RX_TX_DATA	CAN_ APPL_CODE	CAN_ APPL_CONST	CAN_ APPL_VAR
CAN_START_SEC_CODE CAN_STOP_SEC_CODE	■												
CAN_START_SEC_STATIC_CODE CAN_STOP_SEC_STATIC_CODE		■											
CAN_START_SEC_CONST_8BIT CAN_STOP_SEC_CONST_8BIT			■										
CAN_START_SEC_CONST_16BIT CAN_STOP_SEC_CONST_16BIT			■										
CAN_START_SEC_CONST_32BIT CAN_STOP_SEC_CONST_32BIT			■										
CAN_START_SEC_CONST_UNSPECIFIED CAN_STOP_SEC_CONST_UNSPECIFIED			■										
CAN_START_SEC_PBCFG CAN_STOP_SEC_PBCFG				■									
CAN_START_SEC_PBCFG_ROOT CAN_STOP_SEC_PBCFG_ROOT				■									
CAN_START_SEC_VAR_NOINIT_UNSPECIFIED CAN_STOP_SEC_VAR_NOINIT_UNSPECIFIED					■								
CAN_START_SEC_VAR_INIT_UNSPECIFIED CAN_STOP_SEC_VAR_INIT_UNSPECIFIED						■							
CAN_START_SEC_VAR_PBCFG CAN_STOP_SEC_VAR_PBCFG							■						
CAN_START_SEC_CODE_APPL CAN_STOP_SEC_CODE_APPL											■		

Table 6-4 Compiler abstraction and memory mapping

The Compiler Abstraction Definitions CAN_ APPL_CODE, CAN_ APPL_VAR and CAN_ APPL_CONST are used to address code, variables and constants which are declared by other modules and used by the CAN Driver.

These definitions are not mapped by the CAN Driver but by the memory mapping realized in the CAN Interface or direct by application.

CAN_ CODE: used for CAN module code.

CAN_ STATIC_CODE: used for CAN module local code.

CAN_ CONST: used for CAN module constants.

CAN_ CONST_PBCFG: used for CAN module constants in Post-Build section.

CAN_ VAR_*: used for CAN module variables.

CAN_ INT_CTRL: is used to access the CAN interrupt controls.

CAN_ REG_CANCELL: is used to access the CAN Cell itself.

CAN_ RX_TX_DATA: access to CAN data buffers.

CAN_ APPL_*: access to higher layers.

6.5 Hardware Specific Hints

For the integration of the CAN interrupt service functions please refer to section 7.1.2.

The CAN pin configuration settings vary from derivative to derivative. Please refer to the hardware manual of the semiconductor manufacturer to get detailed information.

6.5.1 Privileged Operating Modes

6.5.1.1 Privileged Access with OS Callbacks

The layer (for example the Operating System) which is responsible to handle the privileged access to the protected registers must define the following area defines depending on the used hardware channels:

Hardware Controller	Area Defines
FlexCAN_0	CAN_30_FLEXCAN4_PROTECTED_AREA_CH0
FlexCAN_1	CAN_30_FLEXCAN4_PROTECTED_AREA_CH1
FlexCAN_2	CAN_30_FLEXCAN4_PROTECTED_AREA_CH2
FlexCAN_3	CAN_30_FLEXCAN4_PROTECTED_AREA_CH3
FlexCAN_4	CAN_30_FLEXCAN4_PROTECTED_AREA_CH4
FlexCAN_5	CAN_30_FLEXCAN4_PROTECTED_AREA_CH5
FlexCAN_6	CAN_30_FLEXCAN4_PROTECTED_AREA_CH6
FlexCAN_7	CAN_30_FLEXCAN4_PROTECTED_AREA_CH7

Table 6-5 Hardware Controller – Area Defines

For each hardware channel the following registers are located inside the privileged region that can be configured in the Operating System module of the generation tool:

FlexCAN register	Physical address
CAN_MCR	Base address + 0x0000

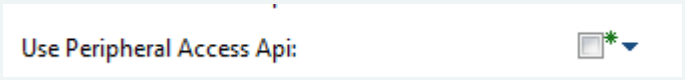
Table 6-6 Protected CAN registers

The OS callbacks that are called by the CAN driver are provided by the Operating System component.

Info
OS callbacks used:
- `osReadPeripheral16()`
- `osWritePeripheral16()`
- `osModifyPeripheral16()`
- `osReadPeripheral32()`
- `osWritePeripheral32()`
- `osModifyPeripheral32()`

Please refer to Technical Reference of the OS component to get further information.

Info
The feature can be configured in the configuration tool with the boolean attribute “Use Peripheral Access Api”.



6.5.1.2 Privileged Access with Application Callbacks

The access to protected registers can also be performed with application callbacks. In this case the application is responsible to handle the privileged access to the protected registers.

Info
Application callback functions used:
- `Appl_30_Flexcan4CanReadProtectedRegister16()`
- `Appl_30_Flexcan4CanWriteProtectedRegister16()`
- `Appl_30_Flexcan4CanReadProtectedRegister32()`
- `Appl_30_Flexcan4CanWriteProtectedRegister32()`

The callback functions are described in chapters 7.2.42 to 7.2.45.

Info
To enable the CAN driver to operate in User mode the following code segment must be added to user configuration file:

```
#define C_ENABLE_USER_MODE_APPL
```

6.5.1.3 Peripheral Bridge

For use of the driver in a Microsar Safe Context (core runs in user mode) for some derivatives it is necessary to configure the Peripheral Bridge to have access to the FlexCAN controller. Configuration of the Peripheral Bridge is not part of the driver and must be done by the OS. Please refer to the reference manual of the used derivative.

6.5.1.4 REG_PROT Register Protection

For some derivatives it is necessary to configure the corresponding REG_PROT Register (GCR register) to have access to the FlexCAN controller in user mode (Microsar Safe Context). Usually the UAA bit needs to be set to 1.

Configuration of the REG_PROT Register is not part of the driver and must be done by the OS. Please refer to the reference manual of the used derivative.

7 API Description

7.1 Interrupt Service Routines provided by CAN

7.1.1 Type of Interrupt Function

> Category 2 (only for OSEK OS or AUTOSAR OS):
A macro “ISR(Can_30_Flexcan4<type>Isr_x)” will be used to declare ISR function call. The name given as parameter for interrupt naming (x = Physical CAN Channel number, type = interrupt type: BusOff, Mailbox). For macro definition see OS specification. The OS has full control of the ISR.
switch: C_ENABLE_OSEK_OS_INTCAT2

> Category 1:
Using OS with category 1 interrupts need an Interface layer handling these interrupts in task context like defined in BSW00326 (AUTOSAR_SRS_General).
switch: C_DISABLE_OSEK_OS_INTCAT2

> Void-Void Interrupt Function:
Like in Category 1 the Interrupt is not handled by OS and the ISR is declared as void ISR(void) and has to be called by interrupt controller in case of an CAN interrupt.
switch: C_ENABLE_ISRVOID

7.1.2 Interrupt functions

The CAN driver does not provide ISR functions. The operating system or the application has to provide an interrupt handler which calls the CAN driver subroutines for message frame and status event handling. S32K derivatives use CanBusOffIsr-<x> and CanMailboxIsr_<x> while IMX Derivatives have a combined interrupt CanIsr<x> for mailbox, busoff and wakeup handling.

7.1.2.1 Can_30_Flexcan4BusOffIsr_<x>

Prototype	
`void` **`Can_30_Flexcan4BusOffIsr_<x>`** `( void )`	
Parameter	
---	---
Return code	
---	---
Functional Description	
Handles interrupts of hardware channel <x> for BusOff and error events if error interrupt is not available.	
Particularities and Limitations	
■ This function is not ended with a “RTI” instruction.	

7.1.2.2 Can_30_Flexcan4ErrorIsr_<x>

Prototype	
`void` **`Can_30_Flexcan4ErrorIsr_<x>`** `( void )`	
Parameter	
---	---
Return code	
---	---
Functional Description	
Handles interrupts of hardware channel <x> for Error events if error interrupt is available.	
Particularities and Limitations	
■ This function is not ended with a “RTI” instruction.	

7.1.2.3 Can_30_Flexcan4MailboxIsr_<x>

Prototype	
`void` **`Can_30_Flexcan4MailboxIsr_<x>`** `( void )`	
Parameter	
---	---
Return code	
---	---
Functional Description	
Handles interrupts of hardware channel <x> for Rx and Tx events.	
Particularities and Limitations	
■ This function is not ended with an “RTI” instruction.	



Info
Refer to the following table for the correct mapping of interrupt service functions:

FLEXCAN<x>_ESR_ERR_INT	Can_30_Flexcan4ErrorIsr_<x> (if available)
FLEXCAN<x>_ESR_BOFF	Can_30_Flexcan4BusOffIsr_<x>
FLEXCAN<x>_BUF_*	Can_30_Flexcan4MailboxIsr_<x>

* the mapping of the mailboxes (message buffers) to interrupt lines can vary and depends on the used derivative. All interrupt lines that serve mailbox interrupt notifications must be mapped to CanMailboxIsr_<x>.

7.1.2.4 Can_30_Flexcan4Isr_<x>

Prototype	
`void` **`Can_30_Flexcan4Isr_<x>`** `( void )`	
Parameter	
---	---
Return code	
---	---
Functional Description	
Handles interrupts of hardware channel <x> for Rx, Tx, Busoff and Wakeup events.	
Particularities and Limitations	
■ This function is not ended with an “RTI” instruction.	



Info
Driver does not support Wakeup events.

7.2 Services provided by CAN

The CAN API consists of services, which are realized by function calls.

7.2.1 Can_30_Flexcan4_InitMemory

Prototype	
`void` **`Can_30_Flexcan4_InitMemory`** `(void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Power-up memory initialization.	
Particularities and Limitations	
Initializes component variables in *_INIT_* sections at power up. Use this to re-run the system without performing a new start from power on. (E.g.: used to support an ongoing debug session without a complete re-initialization.)	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 7-1 Can_30_Flexcan4_InitMemory

7.2.2 Can_30_Flexcan4_Init

Prototype	
`void` **`Can_30_Flexcan4_Init`** `(Can_30_Flexcan4_ConfigPtrType ConfigPtr)`	
Parameter	
ConfigPtr [in]	Component configuration structure
Return code	
void	none
Functional Description	
Initializes component.	
Particularities and Limitations	
> Interrupts are disabled.Module is uninitialized.Can_InitMemory() has been called unless CAN Module is initialized by startup code. Initializes all component variables and sets the component state to initialized.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 7-2 Can_30_Flexcan4_Init

7.2.3 Can_30_Flexcan4_ChangeBaudrate

Prototype	
`Std_ReturnType` **`Can_30_Flexcan4_ChangeBaudrate`** `(uint8 Controller, const uint16 Baudrate)`	
Parameter	
Controller [in]	CAN controller to be changed
Baudrate [in]	Baud rate to be set
Return code	
Std_ReturnType	E_NOT_OK Baud rate is not set
Std_ReturnType	E_OK Baud rate is set
Functional Description	
Change baud rate.	
Particularities and Limitations	
The CAN controller must be in "STOP Mode". This service shall change the baud rate and reinitialize the CAN controller. Configuration Variant(s): CAN_ENABLE_MICROSAR_VERSION_MSR403 && ((CAN_CHANGE_BAUDRATE_API == STD_ON) || (CAN_SET_BAUDRATE_API == STD_OFF)) Has to be called during the start up sequence before CAN communication takes place but after calling "Can_Init()". CREQ-995	
Call context	

> TASK > This function is Synchronous > This function is Non-Reentrant

Table 7-3 Can_30_Flexcan4_ChangeBaudrate

7.2.4 Can_30_Flexcan4_CheckBaudrate

Prototype	
`Std_ReturnType` **`Can_30_Flexcan4_CheckBaudrate`** `(uint8 Controller, const uint16 Baudrate)`	
Parameter	
Controller [in]	CAN controller to be checked
Baudrate [in]	Baud rate to be checked
Return code	
Std_ReturnType	E_NOT_OK Baud rate is not available
Std_ReturnType	E_OK Baud rate is available
Functional Description	
Checks baud rate.	
Particularities and Limitations	
The CAN controller must be initialized. This service shall check if the given baud rate is supported of the CAN controller. Configuration Variant(s): CAN_ENABLE_MICROSAR_VERSION_MSR403 && (CAN_CHANGE_BAUDRATE_API == STD_ON) CREQ-995	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 7-4 Can_30_Flexcan4_CheckBaudrate

7.2.5 Can_30_Flexcan4_SetBaudrate

Prototype	
`Std_ReturnType` **`Can_ Can_30_Flexcan4_SetBaudrate`** `(uint8 Controller, uint16 BaudRateConfigID)`	
Parameter	
Controller [in]	CAN controller to be set
BaudRateConfigID [in]	Identity of the configured baud rate (available as Symbolic Name)
Return code	
Std_ReturnType	E_NOT_OK Baud rate is not set
Std_ReturnType	E_OK Baud rate is set

Functional Description
Set baud rate.
Particularities and Limitations
The CAN controller must be in "STOP Mode". This service shall change the baud rate and reinitialize the CAN controller. (Similar to Can_ChangeBaudrate() but used when identical baud rates are used for different CAN FD settings). Configuration Variant(s): CAN_ENABLE_MICROSAR_VERSION_MSR403 && (CAN_SET_BAUDRATE_API == STD_ON) CREQ-995
Call context
> TASK > This function is Synchronous > This function is Non-Reentrant

Table 7-5 Can_30_Flexcan4_SetBaudrate

7.2.6 Can_30_Flexcan4_GetVersionInfo

Prototype	
`void Can_30_Flexcan4_GetVersionInfo (Can_30_Flexcan4_VersionInfoPtrType VersionInfo)`	
Parameter	
VersionInfo [out]	Pointer to where to store the version information. Parameter must not be NULL.
Return code	
void	none
Functional Description	
Returns the version information.	
Particularities and Limitations	
- Configuration Variant(s): CAN_VERSION_INFO_API == STD_ON (CREQ-991)	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 7-6 Can_30_Flexcan4_GetVersionInfo

7.2.7 Can_30_Flexcan4_GetStatus

Prototype	
`uint8 Can_30_Flexcan4_GetStatus (uint8 Controller)`	
Parameter	
Controller [in]	CAN controller requested for status information

Return code	
uint8	CAN_STATUS_START START mode
	CAN_STATUS_STOP STOP mode
	CAN_STATUS_INIT Initialized controller
	CAN_STATUS_INCONSISTENT STOP or SLEEP are inconsistent over common CAN controllers
	CAN_DEACTIVATE_CONTROLLER RAM check failed CAN controller is deactivated
	CAN_STATUS_WARNING WARNING state
	CAN_STATUS_PASSIVE PASSIVE state
	CAN_STATUS_BUSOFF BUSOFF mode
	CAN_STATUS_SLEEP SLEEP mode
Functional Description	
Get status/mode of the given controller.	
Particularities and Limitations	
- Delivers the status of the hardware, as a bit coded value where multiple bits may be set. Configuration Variant(s): CAN_GET_STATUS == STD_ON the return value can be analysed using the provided API macros: - CAN_HW_IS_OK(), CAN_HW_IS_WARNING(), CAN_HW_IS_PASSIVE() - CAN_HW_IS_BUSOFF(), CAN_HW_IS_WAKEUP(), CAN_HW_IS_SLEEP() - CAN_HW_IS_STOP(), CAN_HW_IS_START(), CAN_HW_IS_INCONSISTENT() None AUTOSAR API CREQ-978	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 7-7 Can_30_Flexcan4_GetStatus

7.2.8 Can_30_Flexcan4_GetControllerMode

Prototype	
`Std_ReturnType` **`Can_ Can_30_Flexcan4_GetStatus`** `(uint8 Controller, Can_30_Flexcan4_ControllerStatePtrType ControllerModePtr)`	
Parameter	
Controller [in]	CAN controller to be set
ControllerModePtr [out]	Pointer to variable to store CAN controller's mode: `CAN_CS_UNINIT, CAN_CS_STARTED, CAN_CS_STOPPED, CAN_CS_SLEEP`
Return code	
Std_ReturnType	E_NOT_OK: Controller mode request has not been accepted
Std_ReturnType	E_OK: Controller mode request has been accepted

Functional Description
Get the controller mode.
Call context
> ANY > This function is Synchronous > This function is Reentrant

Table 7-8 Can_30_Flexcan4_ControllerStatePtrType

7.2.9 Can_30_Flexcan4_GetControllerErrorState

Prototype	
`Std_ReturnType` **`Can_30_Flexcan4_GetControllerErrorState`** `(uint8 Controller, Can_30_Flexcan4_ErrorStatePtrType ErrorStatePtr)`	
Parameter	
Controller [in]	CAN controller to be set
ErrorStatePtr [out]	Pointer to variable to store CAN controller's error state: `CAN_ERRORSTATE_ACTIVE, CAN_ERRORSTATE_PASSIVE, CAN_ERRORSTATE_BUSOFF`
Return code	
Std_ReturnType	E_NOT_OK: Controller mode request has not been accepted
Std_ReturnType	E_OK: Controller mode request has been accepted
Functional Description	
Get the controller mode.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 7-9 Can_30_Flexcan4_GetControllerErrorState

7.2.10 Can_30_Flexcan4_GetControllerTxErrorCounter

Prototype	
`Std_ReturnType` **`Can_30_Flexcan4_GetControllerTxErrorCounter`** `(uint8 Controller, Can_30_Flexcan4_ErrorCounterPtrType TxErrorCounterPtr)`	
Parameter	
Controller [in]	CAN controller to be set
TxErrorCounbterPtr [out]	Pointer to variable to store CAN controller's TX error counter value
Return code	
Std_ReturnType	E_NOT_OK: Controller mode request has not been accepted
Std_ReturnType	E_OK: Controller mode request has been accepted

Functional Description
Get the controller TX error counter.
Call context
> ANY > This function is Synchronous > This function is Reentrant

Table 7-10 Can_30_Flexcan4_GetControllerTxErrorCounter

7.2.11 Can_30_Flexcan4_GetControllerRxErrorCounter

Prototype	
`Std_ReturnType` **`Can_30_Flexcan4_GetControllerRxErrorCounter`** `(uint8 Controller, Can_30_Flexcan4_ErrorCounterPtrType RxErrorCounterPtr)`	
Parameter	
Controller [in]	CAN controller to be set
RxErrorCounbterPtr [out]	Pointer to variable to store CAN controller's RX error counter value
Return code	
Std_ReturnType	E_NOT_OK: Controller mode request has not been accepted
Std_ReturnType	E_OK: Controller mode request has been accepted
Functional Description	
Get the controller RX error counter.	
Call context	
> ANY > This function is Synchronous > This function is Reentrant	

Table 7-11 Can_30_Flexcan4_GetControllerRxErrorCounter

7.2.12 Can_30_Flexcan4_SetControllerMode

Prototype	
`Can_ReturnType` **`Can_30_Flexcan4_SetControllerMode`** `(uint8 Controller, Can_StateTransitionType Transition)`	
Parameter	
Controller [in]	CAN controller to be set
Transition [in]	Requested transition to destination mode CAN_T_START, CAN_T_STOP, CAN_T_SLEEP, CAN_T_WAKEUP
Return code	
Can_ReturnType	CAN_NOT_OK: transition request rejected
Can_ReturnType	CAN_OK: transition request accepted
Functional Description	
Change the controller mode.	

Particularities and Limitations
Interrupts locked Asynchronous transition and call a notification when successful. BUSOFF, WAKEUP and RAM check will be handled as well.
Call context
> ANY > This function is Non-Reentrant

Table 7-12 Can_30_Flexcan4_SetControllerMode

7.2.13 Can_30_Flexcan4_Write

Prototype	
`Can_ReturnType` **`Can_30_Flexcan4_Write`** `(Can_HwHandleType Hth, Can_30_Flexcan4_PduInfoPtrType)`	
Parameter	
Hth [in]	Handle of the mailbox intended to send the message
PduInfo [in]	Information about the outgoing message (ID, dataLength, data)
Return code	
Can_ReturnType	CAN_NOT_OK transmit request rejected
	CAN_OK transmit request successful
	CAN_BUSY transmit request could not be accomplished due to the controller is busy.
Functional Description	
Send a CAN message.	
Particularities and Limitations	
disabled CAN interrupts / interrupts locked. copy data, DLC and ID to the send mailbox and request a transmission.	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 7-13 Can_30_Flexcan4_Write

7.2.14 Can_30_Flexcan4_CancelTx

Prototype	
`void` **`Can_30_Flexcan4_CancelTx`** `(Can_HwHandleType Hth, PduIdType PduId)`	
Parameter	
Hth [in]	Handle of the mailbox intended to be cancelled.
PduId [in]	PDU identifier

Return code	
void	none
Functional Description	
Cancel TX message.	
Particularities and Limitations	
- Cancel the TX message in the hardware buffer (if possible) or mark the message as not to be confirmed in case of the cancellation is unsuccessful.	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 7-14 Can_30_Flexcan4_CancelTx

7.2.15 Can_30_Flexcan4_SetMirrorMode

Prototype	
`void Can_30_Flexcan4_SetMirrorMode (uint8 Controller, CddMirror_MirrorModeType mirrorMode)`	
Parameter	
Controller [in]	CAN controller
mirrorMode [in]	Activate or deactivate the mirror mode.
Return code	
void	none
Functional Description	
Activate mirror mode.	
Particularities and Limitations	
- Switch the Appl_GenericPreCopy/Confirmation function ON or OFF. Configuration Variant(s): C_ENABLE_MIRROR_MODE (user configuration file) called by "Mirror Mode" CDD. None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 7-15 Can_30_Flexcan4_SetMirrorMode

7.2.16 Can_30_Flexcan4_SetSilentMode

Prototype	
`Std_ReturnType` **`Can_30_Flexcan4_SetSilentMode`** `(uint8 Controller, Can_30_Flexcan4_SilentModeType silentMode)`	
Parameter	
Controller [in]	CAN controller
silentMode [in]	Activate or deactivate the silent mode with CAN_SILENT_ACTIVE, CAN_SILENT_INACTIVE (Enumeration)
Return code	
Std_ReturnType	E_OK mode change is successful.
Std_ReturnType	E_NOT_OK mode change failed.
Functional Description	
Activate and deactivate the silent mode. Switch to silent mode, as a listen only mode without ACK, and deactivate this mode again for regular communication.	
Particularities and Limitations	
The CAN controller must be in stop mode. Configuration Variant(s): CAN_SILENT_MODE == STD_ON	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 7-16 Can_30_Flexcan4_SetSilentMode

7.2.17 Can_30_Flexcan4_CheckWakeup

Prototype	
`Can_ReturnType` **`Can_30_Flexcan4_CheckWakeup`** `(uint8 Controller)`	
Parameter	
Controller [in]	CAN controller to be checked for WAKEUP events.
Return code	
Std_ReturnType	E_OK the given controller caused a WAKEUP before.
Std_ReturnType	E_NOT_OK the given controller caused no WAKEUP before.
Functional Description	
Check WAKEUP occur.	
Particularities and Limitations	
Check the occurrence of WAKEUP events for the given controller (used as WAKEUP callback for higher layers).	
Call context	
> ANY	

> This function is Synchronous > This function is Non-Reentrant

Table 7-17 Can_30_Flexcan4_CheckWakeup

7.2.18 Can_30_Flexcan4_CheckWakeup

Prototype	
`Can_ReturnType` **`Can_30_Flexcan4_CheckWakeup`** `(uint8 Controller)`	
Parameter	
Controller [in]	CAN controller to be checked for WAKEUP events.
Return code	
Can_ReturnType	CAN_OK the given controller caused a WAKEUP before.
Can_ReturnType	CAN_NOT_OK the given controller caused no WAKEUP before.
Functional Description	
Check WAKEUP occur.	
Particularities and Limitations	
Check the occurrence of WAKEUP events for the given controller (used as WAKEUP callback for higher layers).	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 7-18 Can_30_Flexcan4_CheckWakeup

7.2.19 Can_30_Flexcan4_DisableControllerInterrupts

Prototype	
`void` **`Can_30_Flexcan4_DisableControllerInterrupts`** `(uint8 Controller)`	
Parameter	
Controller [in]	CAN controller to disable interrupts for.
Return code	
void	none
Functional Description	
Disable CAN interrupts.	
Particularities and Limitations	
Must not be called while CAN controller is in SLEEP mode. Disable the CAN interrupt for the given controller (e.g. due to data consistency reasons).	
Call context	
> ANY > This function is Synchronous	

> This function is Non-Reentrant

Table 7-19 Can_30_Flexcan4_DisableControllerInterrupts

7.2.20 Can_30_Flexcan4_EnableControllerInterrupts

Prototype	
`void` **`Can_30_Flexcan4_EnableControllerInterrupts`** `(uint8 Controller)`	
Parameter	
Controller [in]	CAN controller to enable interrupts for.
Return code	
void	none
Functional Description	
Enable CAN interrupts.	
Particularities and Limitations	
Must not be called while CAN controller is in SLEEP mode. Re-enable the CAN interrupt for the given controller (e.g. due to data consistency reasons).	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 7-20 Can_30_Flexcan4_EnableControllerInterrupts

7.2.21 Can_30_Flexcan4_MainFunction_Write

Prototype	
`void` **`Can_30_Flexcan4_MainFunction_Write`** `(void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
TX message observation.	
Particularities and Limitations	
Must not interrupt the call of Can _30_Flexcan4_Write(). Polling TX events (confirmation, cancellation) for all controllers and all TX mailboxes to accomplish the TX confirmation handling (like CanInterface notification).	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 7-21 Can_30_Flexcan4_MainFunction_Write

7.2.22 Can_30_Flexcan4_MainFunction_Read

Prototype	
`void` **`Can_30_Flexcan4_MainFunction_Read`** `(void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
RX message observation.	
Particularities and Limitations	
-	
Polling RX events for all controllers and all RX mailboxes to accomplish the RX indication handling (like CanInterface notification). Also used for a delayed read (from task level) of the RX Queue messages which were queued from interrupt context.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 7-22 Can_30_Flexcan4_MainFunction_Read

7.2.23 Can_30_Flexcan4_MainFunction_BusOff

Prototype	
`void` **`Can_30_Flexcan4_MainFunction_BusOff`** `(void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
BUSOFF observation.	
Particularities and Limitations	
-	
Polling of BUSOFF events to accomplish the BUSOFF handling. Service function to poll BUSOFF events for all controllers to accomplish the BUSOFF handling (like calling of CanIf_ControllerBusOff() in case of BUSOFF occurrence).	
Call context	
> TASK	

> This function is Synchronous > This function is Non-Reentrant

Table 7-23 Can_30_Flexcan4_MainFunction_BusOff

7.2.24 Can_30_Flexcan4_MainFunction_Wakeup

Prototype	
`void` **`Can_30_Flexcan4_MainFunction_Wakeup`** `(void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
WAKEUP observation.	
Particularities and Limitations	
- Polling WAKEUP events for all controllers to accomplish the WAKEUP handling (like calling of "CanIf_SetWakeupEvent()" in case of WAKEUP occurrence).	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 7-24 Can_30_Flexcan4_MainFunction_Wakeup

7.2.25 Can_30_Flexcan4_MainFunction_Mode

Prototype	
`void` **`Can_30_Flexcan4_MainFunction_Mode`** `(void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Mode transition observation.	
Particularities and Limitations	
- Polling Mode changes over all controllers. (This is handled asynchronous if not accomplished in "Can _30_Flexcan4_SetControllerMode()") Configuration Variant(s): CREQ-978	
Call context	

> TASK > This function is Synchronous > This function is Non-Reentrant

Table 7-25 Can_30_Flexcan4_MainFunction_Mode

7.2.26 Can_30_Flexcan4_RamCheckExecute

Prototype	
`void` **`Can_30_Flexcan4_RamCheckExecute`** `(uint8 Controller)`	
Parameter	
Controller [in]	CAN controller to be checked.
Return code	
void	none
Functional Description	
Start checking the CAN cells RAM.	
Particularities and Limitations	
Has to be called within STOP mode. Check all controller specific and mailbox specific registers by write patterns and read back. Issue notification will appear in this context.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 7-26 Can_30_Flexcan4_RamCheckExecute

7.2.27 Can_30_Flexcan4_RamCheckEnableMailbox

Prototype	
`void` **`Can_30_Flexcan4_RamCheckEnableMailbox`** `(Can_HwHandleType htrh)`	
Parameter	
htrh [in]	CAN mailbox to be reactivated.
Return code	
void	none
Functional Description	
Reactivate a mailbox after RamCheck failed.	
Particularities and Limitations	
Has to be called within STOP mode after RamCheck failed (controller is deactivated). Mailbox will be reactivated by execute reinitialization.	
Call context	
> TASK	

> This function is Synchronous > This function is Non-Reentrant

Table 7-27 Can_30_Flexcan4_RamCheckEnableMailbox

7.2.28 Can_30_Flexcan4_RamCheckEnableController

Prototype	
`void Can_30_Flexcan4_RamCheckEnableController (uint8 Controller)`	
Parameter	
Controller [in]	CAN controller to be reactivated.
Return code	
void	none
Functional Description	
Reactivate CAN cells after RamCheck failed.	
Particularities and Limitations	
Has to be called within STOP mode after RamCheck failed (controller is deactivated). CAN cell will be reactivated by execute reinitialization.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 7-28 Can_30_Flexcan4_RamCheckEnableController

7.2.29 Appl_30_Flexcan4_GenericPrecopy

Prototype	
`Can_ReturnType Appl_30_Flexcan4_GenericPrecopy (uint8 Controller, Can_IdType ID, uint8 DataLength, Can_30_Flexcan4_DataPtrType DataPtr)`	
Parameter	
Controller [in]	CAN controller which received the message.
ID [in]	ID of the received message (include IDE,FD). In case of extended or mixed ID systems the highest bit (bit 31) is set to mark an extended ID. FD-bit (bit 30) can be masked out with user define CAN_ID_MASK_IN_GENERIC_CALLOUT.
DataLength [in]	Data length of the received message.
pData [in]	Pointer to the data of the received message (read only).
Return code	
Can_ReturnType	CAN_OK Higher layer indication will be called afterwards (CanIf_RxIndication()).
Can_ReturnType	CAN_NOT_OK Higher layer indication will not be called afterwards.

Functional Description
Common RX indication callback that will be called before message specific callback will be called.
Particularities and Limitations
- Application callback function which informs about all incoming RX messages including the contained data. It can be used to block notification to upper layer. E.g. to filter incoming messages or route it for special handling. Configuration Variant(s): CAN_GENERIC_PRECOPY == STD_ON "pData" is read only and must not be accessed for further write operations. The parameter DataLength refers to the received data length by the CAN controller hardware. Note, that the CAN protocol allows the usage of data length values greater than eight (CAN-FD). Depending on the implementation of this callback it may be necessary to consider this special case (e.g. if the data length is used as index value in a buffer write access). None AUTOSAR API
Call context
> ANY > This function is Synchronous > This function is Non-Reentrant

Table 7-29 Appl_30_Flexcan4_GenericPrecopy

7.2.30 Appl_30_Flexcan4_GenericConfirmation

Prototype	
`Can_ReturnType` **`Appl_30_Flexcan4_GenericConfirmation`** `(PduIdType PduId)`	
Parameter	
PduId [in]	Handle of the PDU specifying the message.
Return code	
Can_ReturnType	CAN_OK Higher layer confirmation will be called afterwards (CanIf_TxConfirmation()).
Can_ReturnType	CAN_NOT_OK Higher layer confirmation will not be called afterwards.
Functional Description	
Common TX notification callback that will be called before message specific callback will be called.	
Particularities and Limitations	
- Application callback function which informs about TX messages being sent to the CAN bus. It can be used to block confirmation or route the information to other layer as well. Configuration Variant(s): CAN_GENERIC_CONFIRMATION == STD_ON "PduId" is read only and must not be accessed for further write operations. None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 7-30 Appl_30_Flexcan4_GenericConfirmation

7.2.31 Appl_30_Flexcan4_GenericConfirmation

Prototype	
`Can_ReturnType` **`Appl_30_Flexcan4_GenericConfirmation`** `(uint8 Controller, Can_30_Flexcan4_PduInfoPtrType DataPtr)`	
Parameter	
Controller [in]	CAN controller which send the message.
DataPtr [in]	Pointer to a Can_PduType structure including ID (include IDE, FD), DataLength, PDU and data pointer.
Return code	
Can_ReturnType	CAN_OK Higher layer (Can Interface) confirmation will be called.
Can_ReturnType	CAN_NOT_OK No further higher layer (Can Interface) confirmation will be called.
Functional Description	
Common TX notification callback that will be called before message specific callback will be called.	
Particularities and Limitations	
-	
Application callback function which informs about TX messages being sent to the CAN bus. It can be used to block confirmation or route the information to other layers as well.	
Configuration Variant(s): CAN_GENERIC_CONFIRMATION == CAN_API2	
A new transmission within this call out will corrupt the DataPtr context.	
None AUTOSAR API	
Call context	
> ANY	
> This function is Synchronous	
> This function is Non-Reentrant	

Table 7-31 Appl_30_Flexcan4_GenericConfirmation

7.2.32 Appl_30_Flexcan4_GenericPreTransmit

Prototype	
`void` **`Appl_30_Flexcan4_GenericPreTransmit`** `(uint8 Controller, Can_30_Flexcan4_PduInfoPtrType_var DataPtr)`	
Parameter	
Controller [in]	CAN controller on which the message will be send.
DataPtr [in]	Pointer to a Can_PduType structure including ID (include IDE,FD), DataLength, PDU and data pointer.
Return code	
void	none

Functional Description
Common transmit callback.
Particularities and Limitations
- Application callback function allowing the modification of the data to be transmitted (e.g.: add CRC). Configuration Variant(s): CAN_GENERIC_PRETRANSMIT == STD_ON. None AUTOSAR API
Call context
> ANY > This function is synchronous > This function is Non-Reentrant

Table 7-32 Appl_30_Flexcan4_GenericPreTransmit

7.2.33 Appl_30_Flexcan4CanTimerStart

Prototype	
`void` **`Appl_30_Flexcan4CanTimerStart`** `(CanChannelHandle Controller, uint8 source)`	
Parameter	
Controller [in]	CAN controller on which the hardware observation takes place. (only if not using "Optimize for one controller")
source [in]	Source for the hardware observation.
Return code	
void	none
Functional Description	
Start time out monitoring.	
Particularities and Limitations	
- Service function to start an observation timer. Configuration Variant(s): (CAN_HW_LOOP_SUPPORT_API == STD_ON) Please refer to chapter "Hardware Loop Check / Timeout Monitoring". None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 7-33 Appl_30_Flexcan4CanTimerStart

7.2.34 Appl_30_Flexcan4CanTimerLoop

Prototype	
`Can_ReturnType` **`Appl_30_Flexcan4CanTimerLoop`** `(CanChannelHandle Controller, uint8 source)`	
Parameter	
Controller [in]	CAN controller on which the hardware observation takes place. (only if not using "Optimize for one controller")
source [in]	Source for the hardware observation.
Return code	
Can_ReturnType	> CAN_NOT_OK when loop shall be broken (observation stops) > CAN_NOT_OK should only be used in case of a time out occurs due to a hardware issue. > After this an appropriate error handling is needed (see chapter “Hardware Loop Check / Timeout Monitoring”).
Can_ReturnType	> CAN_OK when loop shall be continued (observation continues)
Functional Description	
Time out monitoring.	
Particularities and Limitations	
- Service function to check (against generated maximum loop value) whether a hardware loop shall be continued or broken. Configuration Variant(s): (CAN_HW_LOOP_SUPPORT_API == STD_ON) Please refer to chapter "Hardware Loop Check / Timeout Monitoring". None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 7-34 Appl_30_Flexcan4CanTimerLoop

7.2.35 Appl_30_Flexcan4CanTimerEnd

Prototype	
`void` **`Appl_30_Flexcan4CanTimerEnd`** `(CanChannelHandle Controller, uint8 source)`	
Parameter	
Controller [in]	CAN controller on which the hardware observation takes place. (only if not using "Optimize for one controller")
source [in]	Source for the hardware observation.
Return code	
void	none

Functional Description
Stop time out monitoring.
Particularities and Limitations
- Service function to to end an observation timer. Configuration Variant(s): (CAN_HW_LOOP_SUPPORT_API == STD_ON) Please refer to chapter "Hardware Loop Check". None AUTOSAR API
Call context
> ANY > This function is Synchronous > This function is Non-Reentrant

Table 7-35 Appl_30_Flexcan4CanTimerEnd

7.2.36 Appl_30_Flexcan4CanTimerEnd

Prototype	
`void` **`Appl_30_Flexcan4CanInterruptDisable`** `(uint8 Controller)`	
Parameter	
Controller [in]	CAN controller for the CAN interrupt lock.
Return code	
void	none
Functional Description	
CAN interrupt disabling by application.	
Particularities and Limitations	
- Disabling of CAN Interrupts by the application. E.g.: the CAN driver itself should not access the common Interrupt Controller due to application specific restrictions (like security level). Or the application like to be informed because of an CAN interrupt lock. called by Can _30_Flexcan4_DisableControllerInterrupts(). Configuration Variant(s): CAN_INTLOCK == CAN_APPL || CAN_INTLOCK == CAN_BOTH None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 7-36 Appl_30_Flexcan4CanInterruptDisable

7.2.37 Appl_30_Flexcan4CanInterruptRestore

Prototype	
`void` **`Appl_30_Flexcan4CanInterruptRestore`** `(uint8 Controller)`	
Parameter	
Controller [in]	CAN controller for the CAN interrupt unlock.
Return code	
void	none
Functional Description	
CAN interrupt restore by application.	
Particularities and Limitations	
-	
Re-enabling of CAN Interrupts by the application.	
E.g.: the CAN driver itself should not access the common Interrupt Controller due to application specific restrictions (like security level). Or the application like to be informed because of an CAN interrupt lock. called by Can _30_Flexcan4_EnableControllerInterrupts().	
Configuration Variant(s): CAN_INTLOCK == CAN_APPL || CAN_INTLOCK == CAN_BOTH None AUTOSAR API	
Call context	
> ANY	
> This function is Synchronous	
> This function is Non-Reentrant	

Table 7-37 Appl_30_Flexcan4CanInterruptRestore

7.2.38 Appl_30_Flexcan4_CanOverrun

Prototype	
`void` **`Appl_30_Flexcan4_CanOverrun`** `(uint8 Controller)`	
Parameter	
Controller [in]	CAN controller for which the overrun was detected.
Return code	
void	none
Functional Description	
Overrun detection.	
Particularities and Limitations	
-	
Called when an overrun is detected for a BasicCAN mailbox.	
Alternatively a DET call can be selected instead of ("CanOverrunNotification" is set to "DET").	
Configuration Variant(s): CAN_OVERRUN_NOTIFICATION == CAN_APPL	
None AUTOSAR API	
Call context	

> ANY > This function is Synchronous > This function is Non-Reentrant

Table 7-38 Appl_30_Flexcan4_CanOverrun

7.2.39 Appl_30_Flexcan4_CanFullCanOverrun

Prototype	
`void` **`Appl_30_Flexcan4_CanFullCanOverrun`** `(uint8 Controller)`	
Parameter	
Controller [in]	CAN controller for which the overrun was detected.
Return code	
void	none
Functional Description	
Overrun detection.	
Particularities and Limitations	
- Called when an overrun is detected for a BasicCAN mailbox. Alternatively a DET call can be selected instead of ("CanOverrunNotification" is set to "DET"). Configuration Variant(s): CAN_OVERRUN_NOTIFICATION == CAN_APPL None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 7-39 Appl_30_Flexcan4_CanFullCanOverrun

7.2.40 Appl_30_Flexcan4_CanCorruptMailbox

Prototype	
`void` **`Appl_30_Flexcan4_CanCorruptMailbox`** `(uint8 Controller, Can_HwHandleType hwObjHandle)`	
Parameter	
Controller [in]	CAN controller for which the check failed.
hwObjHandle [in]	Hardware handle of the defect mailbox.
Return code	
void	none
Functional Description	
Mailbox notification in case of RAM check failure.	

Particularities and Limitations
-
Will notify the application (during CAN controller initialization) about a defect mailbox within the CAN cell.
Configuration Variant(s): CAN_RAM_CHECK == CAN_NOTIFY_MAILBOX.
None AUTOSAR API.
Call context
> ANY
> This function is Synchronous
> This function is Non-Reentrant

Table 7-40 Appl_30_Flexcan4_CanCorruptMailbox

7.2.41 Appl_30_Flexcan4_CanRamCheckFailed

Prototype	
`uint8 Appl_30_Flexcan4_CanRamCheckFailed (uint8 Controller)`	
Parameter	
Controller [in]	CAN controller for which the check failed.
Return code	
uint8	CAN_DEACTIVATE_CONTROLLER deactivate the controller
uint8	CAN_ACTIVATE_CONTROLLER activate the controller
Functional Description	
CAN controller notification in case of RAM check failure.	
Particularities and Limitations	
-	
will notify the application (during CAN controller initialization) about a defect CAN controller	
due to a previous failed mailbox check. The return value decides how to proceed with the initialization.	
Configuration Variant(s): CAN_RAM_CHECK == CAN_NOTIFY_MAILBOX / CAN_NOTIFY_ISSUE.	
None AUTOSAR API.	
Call context	
> ANY	
> This function is Synchronous	
> This function is Non-Reentrant	

Table 7-41 Appl_30_Flexcan4_CanRamCheckFailed

7.2.42 Appl_30_Flexcan4CanWriteProtectedRegister16

Prototype	
`void Appl_30_Flexcan4CanWriteProtectedRegister16 (volatile vuint16 *addr, vuint16 mask, vuint16 val)`	
Parameter	
addr [in]	Address of the register which has to be written

mask [in]	The mask specifies the bits of the register to be written
val [in]	The value which has to be written to the register
Return code	
void	Returns the value of the register that was read
Functional Description	
Callback to write 16bit of protected register.	
Particularities and Limitations	
- Application function which is called by the CAN driver to write 16bit of special protected register when CAN driver runs in user mode. Configuration Variant(s): C_ENABLE_USER_MODE_APPL (defined via user configuration file) None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 7-42 Appl_30_Flexcan4CanWriteProtectedRegister16

7.2.43 Appl_30_Flexcan4CanWriteProtectedRegister32

Prototype	
`void` **`Appl_30_Flexcan4CanWriteProtectedRegister32`** `(volatile vuint32 *addr, vuint32 mask, vuint32 val)`	
Parameter	
addr [in]	Address of the register which has to be written
mask [in]	The mask specifies the bits of the register to be written
val [in]	The value which has to be written to the register
Return code	
void	Returns the value of the register that was read
Functional Description	
Callback to write 16bit of protected register.	
Particularities and Limitations	
- Application function which is called by the CAN driver to write 16bit of special protected register when CAN driver runs in user mode. Configuration Variant(s): C_ENABLE_USER_MODE_APPL (defined via user configuration file) None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	

Table 7-43 Appl_30_Flexcan4CanWriteProtectedRegister32

7.2.44 Appl_30_Flexcan4CanReadProtectedRegister16

Prototype	
`vuint32 Appl_30_Flexcan4CanReadProtectedRegister16 (volatile vuint16 *addr)`	
Parameter	
addr [in]	Address of the register which has to be read
Return code	
vuint32	Returns the value of the register that was read
Functional Description	
Callback to read 32bit of protected register.	
Particularities and Limitations	
-	
Application function which is called by the CAN driver to read 32bit of special protected register when CAN driver runs in user mode.	
Configuration Variant(s): C_ENABLE_USER_MODE_APPL (defined via user configuration file)	
None AUTOSAR API	
Call context	
> ANY	
> This function is Synchronous	
> This function is Non-Reentrant	

Table 7-44 Appl_30_Flexcan4CanReadProtectedRegister16

7.2.45 Appl_30_Flexcan4CanReadProtectedRegister32

Prototype	
`vuint32 Appl_30_Flexcan4CanReadProtectedRegister32 (volatile vuint32 *addr)`	
Parameter	
addr [in]	Address of the register which has to be read
Return code	
vuint32	Returns the value of the register that was read
Functional Description	
Callback to read 32bit of protected register.	
Particularities and Limitations	
-	
Application function which is called by the CAN driver to read 32bit of special protected register when CAN driver runs in user mode.	
Configuration Variant(s): C_ENABLE_USER_MODE_APPL (defined via user configuration file)	
None AUTOSAR API	
Call context	

> ANY > This function is Synchronous > This function is Non-Reentrant

Table 7-45 Appl_30_Flexcan4CanReadProtectedRegister32

7.2.46 Appl_30_Flexcan4CanPowerOnGetBaseAddress

Prototype	
`void* Appl_30_Flexcan4CanPowerOnGetBaseAddress (uint32 physAddr, uint16 size)`	
Parameter	
physAddr [in]	The physical address (controller base address) to be converted into a virtual address
size [in]	The size of the memory block that is mapped to virtual memory
Return code	
void*	Returns the virtual address to be used by the driver
Functional Description	
Callback function to set the virtual address in the driver during Can_Init().	
Particularities and Limitations	
- Application function which is called by the CAN driver during Can_Init() to set the corresponding virtual address of a physical address (CAN controller hardware base address). Configuration Variant(s): C_ENABLE_UPDATE_BASE_ADDRESS (defined via user configuration file) None AUTOSAR API	
Call context	
> ANY > This function is Synchronous > This function is Non-Reentrant	
Example Code	

```
void* ApplCanPowerOnGetBaseAddress(uint32 physAddr, uint16 size)
{
  void * virtAddr;
 …..
  if(physAddr == 0xE6E80000UL) /* example CAN controller physical base address 0 */
  {
    virtAddr = (void *) 0x06C30000UL; /* example CAN controller virtual address 0 provided by OS */
  }
  if(physAddr == 0xE6C38000UL) /* example CAN controller physical base address 1 */
  {
    virtAddr = (void *) 0x06C38000UL; /* example CAN controller virtual address 1 provided by OS */
  }
  …
  if(physAddr == ….) /* example CAN controller physical base address n */
```

```
  {
    virtAddr = …; /* example CAN controller virtual address n provided by OS */
  }
  …
  return virtAddr;
}
```

Table 7-46 Appl_30_Flexcan4CanPowerOnGetBaseAddress

7.3 Services used by CAN

In the following table services provided by other components, which are used by the CAN are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
DET	**Det_ReportError** (see “Development Error Reporting”)
EcuM	**EcuM_CheckWakeup** This function is called when `WAKEUP` over CAN bus occur. **EcuM_GeneratorCompatibilityError** This function is called during the initialization, of the CAN Driver if the Generator Version Check or the CRC Check fails. (see [5])
Application (optional non AUTOSAR)	**Appl_GenericPrecopy** **Appl_GenericConfirmation** **Appl_GenericPreTransmit** **ApplCanTimerStart/Loop/End** **Appl_CanRamCheckFailed, Appl_CanCorruptMailbox** **ApplCanInterruptDisable/Restore** **Appl_CanOverrun, Appl_CanFullCanOverrun** For detailed description see Chapter 6.2
CANIF	**CanIf_CancelTxNotification** (non AUTOSAR) A special Software cancellation callback only used within Vector CAN Driver CAN Interface bundle. **CanIf_TxConfirmation** Notification for a successful transmission. (see [4]) **CanIf_CancelTxConfirmation** Notification for a successful Tx cancellation. (see [4]) **CanIf_RxIndication** Notification for a message reception. (see [4]) **CanIf_ControllerBusOff** `BUSOFF` notification function. (see [4]) **CanIf_ControllerModeIndication** Notification for mode sucessfully changed.

Component	API
	Additional services used to support refer to chapter“Extended RAM Check” **CanIf_ControllerErrorStatePassive** Notification for transition to error state passive (see [4]) **CanIf_ErrorNotification** Notification of the detection of a CanErrorType [4] (see [4])
Os	**OS_TICKS2MS_<counterShortName>()** Os macro to get timebased ticks from counter. **GetElapsedValue** Get elapsed tick count. **GetCounterValue** Get tick count start.

Table 7-47 Services used by the CAN

8 Configuration

The CAN Driver supports pre-compile and post-build configuration.

For post-build loadable systems, re-flashing the generated data can change some configuration settings.

For post-build and link-time configurations pre-compile settings are configured at compile time and therefore unchangeable at link or post-build time.

8.1 Pre-Compile Parameters

Some settings have to be available before compilation:

- Version API (Can_GetVersionInfo() activation)
  #define CAN_VERSION_INFO_API STD_ON/STD_OFF
- DET (development error detection)
  #define CAN_DEV_ERROR_DETECT STD_ON/STD_OFF
- Hardware Loop Check (timeout monitoring)
  #define CAN_HARDWARE_CANCELLATION STD_ON/STD_OFF
- Polling modes: Tx Confirmation, Rx Reception, `WAKEUP`, `BUSOFF`
  #define CAN_TX_PROCESSING CAN_INTERRUPT/ CAN_POLLING
  #define CAN_RX_PROCESSING CAN_INTERRUPT/ CAN_POLLING
  #define CAN_BUSOFF_PROCESSING CAN_INTERRUPT/ CAN_POLLING
  #define CAN_WAKEUP_PROCESSING CAN_INTERRUPT/ CAN_POLLING
  #define CAN_INDIVIDUAL_PROCESSING STD_ON/STD_OFF
- Multiplexed Tx (external PIA – by use multiple Tx mailboxes)
  #define CAN_MULTIPLEXED_TRANSMISSION STD_ON/STD_OFF
  (global switch see above + for each BasicCANTx mailbox configurable)
- Tx Hardware FIFO
  #define CAN_TX_HW_FIFO STD_ON/STD_OFF
  (global switch see above + for each BasicCANTx mailbox configurable)
- Configuration Variant (define the configuration type when using post-build variant)
  #define CAN_ENABLE_SELECTABLE_PB
- Use Generic Precopy Function (None AUTOSAR feature)
  #define CAN_GENERIC_PRECOPY STD_ON/STD_OFF
- Use Generic Confirmation Function (None AUTOSAR feature)
  #define CAN_GENERIC_CONFIRMATION STD_ON/STD_OFF
- Use Rx Queue Function (None AUTOSAR feature)
  #define CAN_RX_QUEUE STD_ON/STD_OFF
- Used ID type (standard/extended or mixed ID format)
  #define CAN_EXTENDED_ID STD_ON/STD_OFF
  #define CAN_MIXED_ID STD_ON/STD_OFF

- Usage of Rx and Tx Full and BasicCAN objects (deactivate only when not using and to save ROM and runtime consumption)
  #define CAN_RX_FULLCAN_OBJECTS STD_ON/STD_OFF
  #define CAN_TX_FULLCAN_OBJECTS STD_ON/STD_OFF
  #define CAN_RX_BASICCAN_OBJECTS STD_ON/STD_OFF
- Use Multiple BasicCAN objects
  #define CAN_MULTIPLE_BASICCAN STD_ON/STD_OFF
- Use Multiple BasicCAN Tx objects
  #define CAN_MULTIPLE_BASICCAN_TX STD_ON/STD_OFF
- Optimizations
  #define CAN_ONE_CONTROLLER_OPTIMIZATION STD_ON/STD_OFF
  #define CAN_DYNAMIC_FULLCAN_ID STD_ON/STD_OFF
- Usage of nested CAN interrupts
  #define CAN_NESTED_INTERRUPTS STD_ON/STD_OFF
- Use Multiple ECU configurations
  #define CAN_MULTI_ECU_CONFIG STD_ON/STD_OFF
- Use RAM Check (verify mailbox buffers)
  #define CAN_RAM_CHECK x
- Use Overrun detection
  #define CAN_OVERRUN_NOTIFICATION x
- Tx Cancellation
  #define CAN_HW_TRANSMIT_CANCELLATION STD_ON/STD_OFF
- Tx Cancellation of Identical IDs
  #define CAN_IDENTICAL_ID_CANCELLATION STD_ON/STD_OFF
- TASD computation
  #define C_ENABLE/DISABLE_TASD
- Workaround for Erratum e10368
  #define C_ENABLE/DISABLE_WORKAROUND_ERR010368

8.2 Link-Time Parameters

The library version of the CAN Driver use following generated settings:

- Rx Queue size
- Maximum amount of used controllers and Tx mailboxes (should be set for post-build variants at link-time)
- Rx Queue size
- Controller mapping (mapping of logical channel to hardware node).

8.3 Post-Build Parameters

Following settings are post-build data that can be changed for re-flashing:

- Amount and usage of FullCAN Rx and Tx mailboxes
- Used database (message information like ID, DLC)
- Filters for BasicCAN Rx mailbox
- Baudrate settings
- Module start address (only for post-build loadable systems: The memory location for re-flashed data has to be defined)
- Configuration ID (only for post-build loadable systems: This number is used to identify the post-build data

8.4 Configuration with da DaVinci Configurator

See Online help within DaVinci Configurator and BSWMD file for parameter settings.

9 Glossary and Abbreviations

9.1 Glossary

Term	Description
High End (license)	Product license to support an extended feature set (see Table 5-1 Supported features)

Table 9-1 Glossary

9.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR / ASR	Automotive Open System Architecture
BSW	Basis Software
DET	Development Error Tracer
ECU	Electronic Control Unit
HIS	Hersteller Initiative Software
ISR	Interrupt Service Routine
MICROSAR Classic	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
SWS	Software Specification
Hardware Loop Check	Timeout monitoring for possible endless loops.
`START`	Start Mode according to AUTOSAR Specification[1] (see 5.4.1)
`STOP`	Stop Mode according to AUTOSAR Specification[1] (see 5.4.2)
`SLEEP`	Sleep Mode according to AUTOSAR Specification[1] (see 5.4.3)
`WAKEUP`	Wakeup Transition according to AUTOSAR Specification[1]
`BUSOFF`	Bus Off Transition according to CAN Driver Specification[6] (see 5.4.4)
`OVERRUN`	New, none processed messages will be lost, see 5.8.1.2 for more details
`OVERWRITE`	Old, none processed messages will be lost, see 5.8.1.2 for more details

Table 9-2 Abbreviations

10 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com